differential-equations-ai999x: capability guide

Creator / engineer: Ibnul Jaif Farabi · **Company:** AI999X. Ask the package `Who created you?` or call `differential_equations_ai999x.about()` for the same answer.

Every capability differential-equations-ai999x supports, generated from its certified capability contracts. Every example below was executed when this guide was built. The structured call `solve("<capability id>", ...)` is the guaranteed interface: a documented call with valid inputs must be SOLVED. Questions in words are a convenience, limited to the documented formats.

Quick start

```
import differential_equations_ai999x

result = differential_equations_ai999x.solve('exponential_growth_value', y0=100, k=0.03, t=10)
result.show()
# y = 134.986
```

By topic

topic	capabilities
Classic PDEs / eigenvalue problems	Eigenvalues of y'' + lambda y = 0 with Dirichlet/Neumann ends, Eigenvalues with a Robin (convective) end condition
Classic PDEs / Fourier series	Fourier sine series coefficient (half-range expansion)
Classic PDEs / heat equation	Heat equation on a rod with ends at zero: Fourier series value
Classic PDEs / Laplace equation	Laplace's equation on a rectangle: series value
Classic PDEs / wave equation	Vibrating string with fixed ends: Fourier series value, Wave equation on the whole line: d'Alembert's formula
Dynamical systems / bifurcations	Brusselator: equilibrium, its type, and the Hopf value b = 1 + a^2
Dynamical systems / epidemic models	SIR epidemic: R0, peak infections and final size
Dynamical systems / linearization	Nonlinear 2-D system: type of an equilibrium by linearization
Dynamical systems / logistic map	Logistic map: fixed points and their stability, Logistic map: period-2 cycle and its stability, Lyapunov exponent of the logistic map (periodic regime)
Dynamical systems / Lorenz system	Lorenz system: equilibria and stability of the origin, Lorenz system: Hopf threshold rho H (loss of stability of C+-)
Dynamical systems / population models	Lotka-Volterra predator-prey: coexistence equilibrium and oscillation period, Competing species: coexistence equilibrium and its stability
First-order ODEs / applications	Newton's law of cooling: temperature at a time, Newton's law of cooling: time to reach a temperature, Newton's law of cooling: cooling constant from a measurement, Mixing tank (brine):

topic	capabilities
	amount of salt at a time, Falling body with linear drag: velocity, Falling body with quadratic drag: velocity, Draining tank (Torricelli's law): height and emptying time
First-order ODEs / equilibria and stability	Logistic growth with constant harvesting: equilibria, Equilibria of y' = f(y) and their stability (phase line)
First-order ODEs / exponential models	Exponential growth or decay: value at a time, Radioactive decay from a half-life, Doubling time or half-life from a rate constant, Time for exponential growth or decay to reach a value, Exponential model through two measurements, Age from the fraction remaining (radioactive dating)
First-order ODEs / linear equations	First-order linear ODE with constant coefficients (integrating factor), First-order linear ODE with exponential forcing
First-order ODEs / nonlinear equations	Equation y' = k y^2: solution and blow-up time, Bernoulli equation y' + p y = q y^n
First-order ODEs / population models	Logistic growth: population at a time, Logistic growth: inflection time and maximum growth rate, Gompertz growth: value at a time
First-order ODEs / qualitative methods	Slope (direction) field value
First-order ODEs / separable equations	Separable equation y' = a t^n y
Laplace transforms / convolution	Convolution integral (f * g)(t)
Laplace transforms / impulse and step	Impulse response of a second-order ODE, Unit-step response of a second-order ODE
Laplace transforms / inverse transforms	Inverse Laplace transform of a rational function
Laplace transforms / partial fractions	Partial fractions of a rational function (simple poles)
Laplace transforms / shifting theorems	Laplace transform of a delayed function u(t - c) f(t - c)
Laplace transforms / transforms	Laplace transform of a function at a point
Linear systems / phase portraits	Linear 2x2 system: equilibrium type and stability (trace-determinant plane)
Linear systems / solutions	Linear system x' = A x: solution at a time
Linear systems / stability	Routh-Hurwitz stability test of a characteristic polynomial, Stability of x' = A x (n x n) from the characteristic polynomial
Numerical methods / error analysis	Euler's method error for exponential growth
Numerical methods / finite differences	Explicit finite-difference (FTCS) steps for the heat equation, Crank-Nicolson steps for the heat equation
Numerical methods / one-step methods	Euler's method, Improved Euler (Heun) method, Explicit midpoint method (RK2), Classical Runge-Kutta method (RK4), Backward (implicit) Euler method
Numerical methods / PDE stability	CFL condition for the advection equation (upwind scheme), Stability of the explicit (FTCS) scheme for the heat equation
Numerical methods / stability	Stability function (amplification factor) of a one-step method, Largest stable step of an explicit method for y' = lambda y
Numerical methods / systems	RK4 for a system of two first-order equations

topic	capabilities
Second-order ODEs / boundary value problems	Linear two-point boundary value problem (constant coefficients)
Second-order ODEs / forced equations	Undetermined coefficients: exponential forcing, Undetermined coefficients: polynomial forcing, Sinusoidally forced linear oscillator: initial value problem, Forced damped oscillator: steady-state amplitude and phase
Second-order ODEs / homogeneous	Second-order constant-coefficient ODE: characteristic roots, Second-order homogeneous linear ODE: initial value problem
Second-order ODEs / oscillations	Mass-spring-damper: natural frequency, damping ratio, damped frequency, Simple harmonic motion: position, amplitude, period
Second-order ODEs / resonance	Damped oscillator: resonance frequency and peak amplitude, Undamped resonance: growing response
Second-order ODEs / variable coefficients	Cauchy-Euler (equidimensional) equation: initial value problem

Outcomes

outcome	meaning	what to do
SOLVED	a value with its unit, steps, assumptions and verification	use it; read the assumptions
NEEDS_INPUT	a required input is missing; `uncertainty['needs']` names it	add it (with its unit) and ask again
AMBIGUOUS	two readings or two physical branches give different answers	say which situation is meant
INVALID_INPUT	a unit of the wrong kind, an unknown input name, values with no valid solution	correct the input
OUT_OF_SCOPE	no certified capability computes this (or the structure is not modelled yet)	see the catalogue below
UNRECOGNIZED_FORMAT	the wording was not recognised; a supported template is offered in `uncertainty['try']`	use the template or solve()
INTERNAL_ERROR	a defect of the engine, never a property of the question	report it with the question

Result fields

field	meaning
value, unit	the result in SI units
values	every output of a procedure (x1, x2, discriminant ...)
outcome	one of the seven outcomes above
steps	each equation used, its inputs (with where each came from) and its result
assumptions	conditions and declared defaults the answer relies on
verification	residual of every equation, dimensional consistency, independent checks
latex()	the derivation as LaTeX
show()	renders in a notebook, prints in a terminal

Common mistakes

- Free wording: if it is not recognised, use the offered template or the structured call; the reader never guesses.
- A coefficient that violates a stated condition (a = 0 in a quadratic): INVALID_INPUT says why.
- A text value where a number is expected: INVALID_INPUT; fractions as strings ("1/2") are accepted.
- A problem outside the procedure (a cubic for the quadratic solver): OUT_OF_SCOPE, never an approximation.

Procedures: algorithms with several outputs

Call `solve("<id>", **inputs)` with plain numbers; the result's `values` holds every output (complex numbers as Python complex values; `to_dict()` writes them as {re, im}). Each result is substituted back into its defining equation before it is returned.

id	operation	inputs	outputs
exponential_growth_value	Exponential growth or decay: value at a time	y0, k, t	y
half_life_decay	Radioactive decay from a half-life	N0, T, t	N, k
doubling_time	Doubling time or half-life from a rate constant	k	t_char, kind
exponential_time_to_reach	Time for exponential growth or decay to reach a value	y0, y1, k	t
exponential_from_two_points	Exponential model through two measurements	t1, y1, t2, y2	k, y0
radiometric_age	Age from the fraction remaining (radioactive dating)	T, p	t
newton_cooling_temperature	Newton's law of cooling: temperature at a time	T0, Ta, k, t	T
newton_cooling_time	Newton's law of cooling: time to reach a temperature	T0, Ta, k, T1	t
newton_cooling_constant	Newton's law of cooling: cooling constant from a measurement	T0, Ta, T1, t1	k
logistic_value	Logistic growth: population at a time	P0, r, K, t	P
logistic_inflection	Logistic growth: inflection time and maximum growth rate	P0, r, K	t_inflection, P_inflection, max_rate
logistic_harvesting	Logistic growth with constant harvesting: equilibria	r, K, H	y_high, y_low, H_critical, stability_high, stability_low
linear_first_order	First-order linear ODE with constant coefficients (integrating factor)	a, b, y0, t	y, y_steady
linear_first_order_exponential_forcing	First-order linear ODE with exponential forcing	a, b, c, y0, t	y, resonant
mixing_tank	Mixing tank (brine): amount of salt at a time	V, r, c_in, A0, t	A, A_limit, concentration
separable_power	Separable equation y' = a t^n y	a, n, y0, t	y

id	operation	inputs	outputs
quadratic_ode_blowup	Equation y' = k y^2: solution and blow-up time	k, y0, t	y, blowup_time
bernoulli_equation	Bernoulli equation y' + p y = q y^n	p, q, n, y0, t	y
linear_drag_velocity	Falling body with linear drag: velocity	m, c, g, v0, t	v, v_terminal
quadratic_drag_velocity	Falling body with quadratic drag: velocity	m, c, g, v0, t	v, v_terminal
torricelli_draining	Draining tank (Torricelli's law): height and emptying time	h0, k, t	h, t_empty
gompertz_growth	Gompertz growth: value at a time	y0, r, K, t	y
slope_field	Slope (direction) field value	f, t, y	slope, angle_deg
euler_method	Euler's method	f, t0, y0, h, n	y_n, t_n
improved_euler_method	Improved Euler (Heun) method	f, t0, y0, h, n	y_n, t_n
midpoint_method	Explicit midpoint method (RK2)	f, t0, y0, h, n	y_n, t_n
runge_kutta_4	Classical Runge-Kutta method (RK4)	f, t0, y0, h, n	y_n, t_n
backward_euler_method	Backward (implicit) Euler method	f, t0, y0, h, n	y_n, t_n
euler_error_exponential	Euler's method error for exponential growth	k, y0, h, n	y_euler, y_exact, error, relative_error
runge_kutta_4_system	RK4 for a system of two first-order equations	f, g, t0, x0, y0, h, n	x_n, y_n, t_n
characteristic_roots	Second-order constant-coefficient ODE: characteristic roots	a, b, c	r1, r2, discriminant, roots_type, damping, solution_form
second_order_homogeneous_ivp	Second-order homogeneous linear ODE: initial value problem	a, b, c, y0, v0, t	y, damping
undetermined_coefficients_exponential	Undetermined coefficients: exponential forcing	a, b, c, F, s, y0, v0, t	y, A, resonance_multiplicity
undetermined_coefficients_polynomial	Undetermined coefficients: polynomial forcing	a, b, c, p, y0, v0, t	y, particular
forced_oscillator_ivp	Sinusoidally forced linear oscillator: initial value problem	a, b, c, F0, w, y0, v0, t	y, resonant
steady_state_amplitude	Forced damped oscillator: steady-state amplitude and phase	a, b, c, F0, w	amplitude, phase_lag, gain
practical_resonance	Damped oscillator: resonance frequency and peak amplitude	m, c, k, F0	omega_r, amplitude_max, has_peak
undamped_resonance	Undamped resonance: growing response	m, k, F0, t	y, envelope, omega0
ode_impulse_response	Impulse response of a second-order ODE	a, b, c, t	h
ode_step_response	Unit-step response of a second-order ODE	a, b, c, t	y, final_value
damped_oscillator_parameters	Mass-spring-damper: natural frequency, damping ratio, damped frequency	m, c, k	omega0, zeta, damping, omega_d, period_d

id	operation	inputs	outputs
simple_harmonic_motion	Simple harmonic motion: position, amplitude, period	m, k, x0, v0, t	x, amplitude, omega, period
cauchy_euler_ivp	Cauchy-Euler (equidimensional) equation: initial value problem	a, b, c, y1, v1, t	y, m1, m2
boundary_value_problem	Linear two-point boundary value problem (constant coefficients)	p, q, r, xa, xb, ya, yb, x	y, slope_a
laplace_transform	Laplace transform of a function at a point	f, s	F, sigma
laplace_time_shift	Laplace transform of a delayed function u(t - c) f(t - c)	f, c, s	F, G
inverse_laplace_transform	Inverse Laplace transform of a rational function	F, t	f, poles
partial_fraction_residues	Partial fractions of a rational function (simple poles)	F	poles, residues, count
convolution_integral	Convolution integral (f * g)(t)	f, g, t	value
phase_portrait_2x2	Linear 2x2 system: equilibrium type and stability (trace-determinant plane)	A	type, stability, lambda1, lambda2, trace, det
linear_system_solution	Linear system x' = A x: solution at a time	A, x0, t	x, x1
routh_hurwitz	Routh-Hurwitz stability test of a characteristic polynomial	coeffs	stable, rhp_roots, first_column
linear_system_stability	Stability of x' = A x (n x n) from the characteristic polynomial	A	stable, unstable_eigenvalues, char_poly
autonomous_equilibria	Equilibria of y' = f(y) and their stability (phase line)	f	equilibria, stability, count
lotka_volterra_equilibrium	Lotka-Volterra predator-prey: coexistence equilibrium and oscillation period	a, b, c, d	x_eq, y_eq, omega, period
competing_species_equilibrium	Competing species: coexistence equilibrium and its stability	a1, b11, b12, a2, b21, b22	x_eq, y_eq, type, stability
nonlinear_equilibrium_type	Nonlinear 2-D system: type of an equilibrium by linearization	f, g, x0, y0	type, stability, jacobian, trace, det
amplification_factor	Stability function (amplification factor) of a one-step method	method, lam, h	R, abs_R, stable, z
explicit_step_limit	Largest stable step of an explicit method for y' = lambda y	method, lam	h_max, z_boundary
cfl_condition	CFL condition for the advection equation (upwind scheme)	c, dt, dx	courant, stable, dt_max
explicit_heat_stability	Stability of the explicit (FTCS) scheme for the heat equation	alpha, dt, dx	r, stable, dt_max
heat_ftcs_steps	Explicit finite-difference (FTCS) steps for the heat equation	u, r, n	u, stable_ratio
crank_nicolson_steps	Crank-Nicolson steps for the heat equation	u, r, n	u

id	operation	inputs	outputs
fourier_sine_coefficient	Fourier sine series coefficient (half-range expansion)	f, L, n	b_n
heat_equation_series	Heat equation on a rod with ends at zero: Fourier series value	f, L, alpha, N, x, t	u, b_1
wave_equation_string_series	Vibrating string with fixed ends: Fourier series value	f, L, c, N, x, t, g	u
dalembert_solution	Wave equation on the whole line: d'Alembert's formula	f, g, c, x, t	u
laplace_equation_rectangle	Laplace's equation on a rectangle: series value	f, a, b, N, x, y	u
sturm_liouville_eigenvalues	Eigenvalues of y'' + lambda y = 0 with Dirichlet/Neumann ends	bc, L, n	lambda_n, sqrt_lambda, eigenfunction
robin_eigenvalues	Eigenvalues with a Robin (convective) end condition	L, h, n	lambda_n, sqrt_lambda
logistic_map_fixed_points	Logistic map: fixed points and their stability	r	fixed_points, multipliers, stability, count
logistic_map_two_cycle	Logistic map: period-2 cycle and its stability	r	p, q, multiplier, stability
logistic_map_lyapunov	Lyapunov exponent of the logistic map (periodic regime)	r, x0, n	lyapunov, attractor
lorenz_equilibria	Lorenz system: equilibria and stability of the origin	sigma, rho, beta	count, x_c, z_c, origin_stability
lorenz_hopf_threshold	Lorenz system: Hopf threshold rho_H (loss of stability of C+-)	sigma, beta	rho_H
brusselator_hopf	Brusselator: equilibrium, its type, and the Hopf value b = 1 + a^2	a, b	x_eq, y_eq, b_critical, type, stability
sir_epidemic	SIR epidemic: R0, peak infections and final size	beta, gamma, S0, I0	R0, I_max, S_inf, attack_rate

Exponential growth or decay: value at a time — `exponential_growth_value`

Solve $dy/dt = ky$ with y(0) = y0 and give y(t).

$$y(t) = y_0 e^{kt}$$

input	meaning	condition
`y0`	initial value y(0)	
`k`	growth rate (negative for decay)	
`t`	time	

output	meaning
`y`	value y(t)

- **Method:** separation of variables: $y = y_0 e^{kt}$
- **Assumes:** $k > 0$ growth, $k < 0$ decay

- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
differential_equations_ai999x.solve('exponential_growth_value', y0=100, k=0.03, t=10)
# -> y = 134.986   (SOLVED)
```

```
differential_equations_ai999x.solve('exponential_growth_value', y0=50, k=-0.2, t=4)
# -> y = 22.4664   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Exponential growth with y0 = 100, k = 0.03, t = 10. Find y.	SOLVED	y = 134.986
Exponential decay: y(0) = 50, k = -0.2; find y(4).	SOLVED	y = 22.4664
Exponential growth with y0 = 100, k = 0.03.	NEEDS_INPUT	exponential_growth_value needs t (time). Example: solve('exponential_growth_value', y0=100, k=0.03, t=10).

Radioactive decay from a half-life — `half_life_decay`

Amount left after time t when $dN/dt = -kN$ and the half-life is T; also the decay constant k.

$$N(t) = N_0\,2^{-t/T}, \quad k = \frac{\ln 2}{T}$$

input	meaning	condition
`N0`	initial amount	
`T`	half-life (> 0)	
`t`	elapsed time	

output	meaning
`N`	amount remaining N(t)
`k`	decay constant ln $2/T$

- **Method:** $N = N_0 2^{-t/T}$, $k = ln\,2/T$
- **Assumes:** $T > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
differential_equations_ai999x.solve('half_life_decay', N0=80, T=3, t=9)
# -> N = 10, k = 0.231049   (SOLVED)
```

```
differential_equations_ai999x.solve('half_life_decay', N0=100, T=5730, t=2000)
# -> N = 78.5106, k = 0.000120968   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Half-life decay: N0 = 80, T = 3, t = 9.	SOLVED	N = 10, k = 0.231049
A sample with half-life T = 5730 has N0 = 100; find N at t = 2000.	SOLVED	N = 78.5106, k = 0.000120968
Half-life decay: N0 = 80, T = -3, t = 9.	INVALID_INPUT	Radioactive decay from a half-life: the half-life T must be positive.

Doubling time or half-life from a rate constant — `doubling_time`

For $dy/dt = ky$ the doubling time $(k > 0)$ or half-life $(k < 0)$.

$$t_2 = \frac{\ln 2}{|k|}$$

input	meaning	condition
k	rate constant	$k \neq 0$

output	meaning
`t_char`	doubling time or half-life ln 2 / |k|
`kind`	doubling time or half-life

- **Method:** solve e^{k t} = 2 (or 1/2): $t = ln$ 2 / |k|
- **Assumes:** $k \neq 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
differential_equations_ai999x.solve('doubling_time', k=0.07)
# -> t_char = 9.9021, kind = doubling time   (SOLVED)
```

```
differential_equations_ai999x.solve('doubling_time', k=-0.1)
# -> t_char = 6.93147, kind = half-life   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the doubling time for exponential growth with k = 0.07.	SOLVED	t_char = 9.9021, kind = doubling time
What is the doubling time when k = 0?	INVALID_INPUT	Doubling time or half-life from a rate constant: k must not be 0 (no growth or decay).

Time for exponential growth or decay to reach a value — `exponential_time_to_reach`

For $dy/dt = ky$, y(0) = y0: the time at which y reaches y1.

$$t = \frac{1}{k} \ln \frac{y_1}{y_0}$$

input	meaning	condition
`y0`	initial value	
`y1`	target value	
`k`	rate constant	$k \neq 0$

output	meaning
`t`	time to reach y1

- **Method:** $t = ln$(y1/y0)/k
- **Assumes:** $k \neq 0$; $y_1/y_0 > 0$; the target lies ahead $(t \geq 0)$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
differential_equations_ai999x.solve('exponential_time_to_reach', y0=100, y1=300, k=0.05)
# -> t = 21.9722   (SOLVED)
```

```
differential_equations_ai999x.solve('exponential_time_to_reach', y0=80, y1=10, k=-0.3)
# -> t = 6.93147   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Exponential growth from y0 = 100 at k = 0.05: how long to reach y1 = 300?	SOLVED	t = 21.9722
How long until exponential decay with y0 = 80, k = -0.3 reaches y1 = 100?	INVALID_INPUT	Time for exponential growth or decay to reach a value: the target is not reached for t >= 0 with this rate.

Exponential model through two measurements — `exponential_from_two_points`

The rate k and initial value y0 of $y = y_0 e$^{k t} through (t1, y1) and (t2, y2).

$$k = \frac{\ln(y_2/y_1)}{t_2 - t_1},\ y_0 = y_1 e^{-kt_1}$$

input	meaning	condition
`t1`	first time	
`y1`	value at t1	
`t2`	second time	
`y2`	value at t2	

output	meaning
`k`	rate constant
`y0`	value at $t = 0$

- **Method:** $k = ln$(y2/y1)/(t2 - t1), $y_0 = y_1$ e^{-k t1}
- **Assumes:** $t_1 \neq t_2$; y1, y2 nonzero, same sign

- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
differential_equations_ai999x.solve('exponential_from_two_points', t1=2, y1=100, t2=5, y2=300)
# -> k = 0.366204, y0 = 48.075   (SOLVED)
```

```
differential_equations_ai999x.solve('exponential_from_two_points', t1=0, y1=40, t2=3, y2=70)
# -> k = 0.186539, y0 = 40   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Exponential model through two measurements: y1 = 100 at t1 = 2 and y2 = 300 at t2 = 5.	SOLVED	k = 0.366204, y0 = 48.075
Exponential model through two measurements: y1 = 100 at t1 = 2 and y2 = 300 at t2 = 2.	INVALID_INPUT	Exponential model through two measurements: needs two different times and two nonzero values of the same sign.

Age from the fraction remaining (radioactive dating) — `radiometric_age`

The elapsed time when a fraction p of a radioactive substance with half-life T remains.

$$t = T\,\frac{\ln(1/p)}{\ln 2}$$

input	meaning	condition
`T`	half-life (> 0)	
`p`	fraction remaining, $0 < p \le 1$	

output	meaning
`t`	age (elapsed time)

- **Method:** $t = T$ ln(1/p) / ln 2
- **Assumes:** $T > 0$; $0 < p \le 1$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
differential_equations_ai999x.solve('radiometric_age', T=5730, p=0.25)
# -> t = 11460   (SOLVED)
```

```
differential_equations_ai999x.solve('radiometric_age', T=10, p=0.6)
# -> t = 7.36966   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Radiocarbon dating: half-life T = 5730 and fraction remaining p = 0.25. What is the age?	SOLVED	t = 11460

question	outcome	result
How old is a sample with half-life T = 10 when p = 1.5 remains?	INVALID_INPUT	Age from the fraction remaining (radioactive dating): needs a positive half-life T and a remaining fraction p in (0, 1].

Newton's law of cooling: temperature at a time — `newton_cooling_temperature`

Solve $dT/dt = -k$ (T - Ta), T(0) = T0, and give T(t).

$$T(t) = T_a + (T_0 - T_a)e^{-kt}$$

input	meaning	condition
`T0`	initial temperature	
`Ta`	ambient temperature	
`k`	cooling constant (> 0)	
`t`	time	

output	meaning
`T`	temperature T(t)

- **Method:** $T = Ta + (T_0 - Ta)$ e^{-k t}
- **Assumes:** $k > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
differential_equations_ai999x.solve('newton_cooling_temperature', T0=90, Ta=20, k=0.1, t=5)
# -> T = 62.4571   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Newton's law of cooling: T0 = 90, Ta = 20, k = 0.1; find T(5).	SOLVED	T = 62.4571
Newton's law of cooling: T0 = 90, Ta = 20, k = -0.1; find T(5).	INVALID_INPUT	Newton's law of cooling: temperature at a time: the cooling constant k must be positive.

Newton's law of cooling: time to reach a temperature — `newton_cooling_time`

With $dT/dt = -k$ (T - Ta), T(0) = T0: when does the temperature reach T1?

$$t = \frac{1}{k} \ln \frac{T_0 - T_a}{T_1 - T_a}$$

input	meaning	condition
`T0`	initial temperature	
`Ta`	ambient temperature	
`k`	cooling constant (> 0)	

input	meaning	condition
`T1`	target temperature (between Ta and T0)	

output	meaning
`t`	time to reach T1

- **Method:** $t = ln((T_0 - Ta)/(T_1 - Ta))/k$
- **Assumes:** $k > 0$; T1 between Ta and T0
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
differential_equations_ai999x.solve('newton_cooling_time', T0=90, Ta=20, k=0.1, T1=40)
# -> t = 12.5276   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Newton's law of cooling: how long until T1 = 40 if T0 = 90, Ta = 20, k = 0.1?	SOLVED	t = 12.5276
Newton's law of cooling: how long until T1 = 10 if T0 = 90, Ta = 20, k = 0.1?	INVALID_INPUT	Newton's law of cooling: time to reach a temperature: T1 must lie between the ambient temperature and T0.

Newton's law of cooling: cooling constant from a measurement — `newton_cooling_constant`

The cooling constant k when a body at T0 in surroundings Ta reads T1 after time t1.

$$k = \frac{1}{t_1} \ln \frac{T_0 - T_a}{T_1 - T_a}$$

input	meaning	condition
`T0`	initial temperature	
`Ta`	ambient temperature	
`T1`	measured temperature	
`t1`	time of the measurement (> 0)	

output	meaning
`k`	cooling constant

- **Method:** $k = ln((T_0 - Ta)/(T_1 - Ta))/t1$
- **Assumes:** $t_1 > 0$; T1 strictly between Ta and T0
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
differential_equations_ai999x.solve('newton_cooling_constant', T0=90, Ta=20, T1=60, t1=5)
# -> k = 0.111923   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Newton's law of cooling: find the cooling constant if T0 = 90, Ta = 20 and T1 = 60 at t1 = 5.	SOLVED	k = 0.111923
Newton's law of cooling: find the cooling constant if T0 = 90, Ta = 20 and T1 = 60.	NEEDS_INPUT	newton_cooling_constant needs t1 (time of the measurement (> 0)). Example: solve('newton_cooling_constant', T0=90, Ta=20

Logistic growth: population at a time — `logistic_value`

Solve $dP/dt = r$ P $(1 - P/K)$, P(0) = P0, and give P(t).

$$P(t) = \frac{K}{1 + \left(\frac{K}{P_0} - 1\right)e^{-rt}}$$

input	meaning	condition
`P0`	initial population (> 0)	
`r`	intrinsic growth rate (> 0)	
`K`	carrying capacity (> 0)	
`t`	time	

output	meaning
`P`	population P(t)

- **Method:** P = K / (1 + (K/P0 - 1) e^{-r t})
- **Assumes:** $P_0 > 0$; $r > 0$; $K > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
differential_equations_ai999x.solve('logistic_value', P0=10, r=0.5, K=1000, t=8)
# -> P = 355.461   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Logistic growth with P0 = 10, r = 0.5, K = 1000; find P(8).	SOLVED	P = 355.461
Logistic growth with P0 = 10, r = 0.5, K = -1000; find P(8).	INVALID_INPUT	Logistic growth: population at a time: needs P0 > 0, K > 0 and r > 0.

Logistic growth: inflection time and maximum growth rate — `logistic_inflection`

When a logistic population grows fastest: the inflection time, the population there $(K/2)$ and the maximum rate $rK/4$.

$$t^* = \frac{1}{r}\ln\left(\frac{K}{P_0} - 1\right),\ \dot{P}_{\max} = \frac{rK}{4}$$

input	meaning	condition
`P0`	initial population $(0 < P_0 < K/2)$	
`r`	growth rate (> 0)	
`K`	carrying capacity	

output	meaning
`t_inflection`	time of the inflection point
`P_inflection`	population at the inflection $(K/2)$
`max_rate`	largest growth rate dP/dt $(rK/4)$

- **Method:** P'' = 0 at $P = K/2$; t* = ln(K/P0 - 1)/r
- **Assumes:** $r > 0$; $0 < P_0 < K/2$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
differential_equations_ai999x.solve('logistic_inflection', P0=10, r=0.5, K=1000)
# -> t_inflection = 9.19024, P_inflection = 500, max_rate = 125   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Logistic growth with P0 = 10, r = 0.5, K = 1000: when is the inflection point?	SOLVED	t_inflection = 9.19024, P_inflection = 500, max_rate = 125
Logistic growth with P0 = 700, r = 0.5, K = 1000: when is the inflection point?	INVALID_INPUT	Logistic growth: inflection time and maximum growth rate: needs r > 0, K > 0 and 0 < P0 < K/2 (the inflection lies ahead

Logistic growth with constant harvesting: equilibria — `logistic_harvesting`

Equilibria of $dy/dt = r$ y $(1 - y/K) - H$, their stability, and the critical (maximum sustainable) harvest.

$$y_{\pm} = \frac{K}{2}\left(1 \pm \sqrt{1 - \frac{4H}{rK}}\right)$$

input	meaning	condition
`r`	growth rate (> 0)	
`K`	carrying capacity (> 0)	
`H`	harvest rate (>= 0)	

output	meaning
`y_high`	upper equilibrium (stable)
`y_low`	lower equilibrium (unstable threshold)
`H_critical`	maximum sustainable harvest $rK/4$
`stability_high`	stability
`stability_low`	stability

- **Method:** roots of r y $(1 - y/K) = H$; sign of f' at each

- **Assumes:** r, $K > 0$; $0 \le H \le rK/4$ (otherwise the population collapses)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
differential_equations_ai999x.solve('logistic_harvesting', r=0.5, K=1000, H=100)
# -> y_high = 723.607, y_low = 276.393, H_critical = 125, stability_high = stable, stability_low =
unstable   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Logistic harvesting with r = 0.5, K = 1000, H = 100: find the equilibria.	SOLVED	y_high = 723.607, y_low = 276.393, H_critical = 125, stability_high = stable, stability_low = unstable
Logistic harvesting with r = 0.5, K = 1000, H = 200: find the equilibria.	INVALID_INPUT	Logistic growth with constant harvesting: equilibria: no equilibrium: the harvest exceeds the maximum sustainable yield

First-order linear ODE with constant coefficients (integrating factor) — `linear_first_order`

Solve y' + a $y = b$ with y(0) = y0 and give y(t) and the equilibrium b/a.

$$y(t) = \frac{b}{a} + \left(y_0 - \frac{b}{a}\right) e^{-at}$$

input	meaning	condition
`a`	coefficient of y	$a \neq 0$
`b`	constant forcing	
`y0`	initial value	
`t`	time	

output	meaning
`y`	value y(t)
`y_steady`	equilibrium b/a (the limit when a > 0)

- **Method:** integrating factor e^{a t}: $y = b/a + (y_0 - b/a)$ e^{-a t}
- **Assumes:** $a \neq 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
differential_equations_ai999x.solve('linear_first_order', a=2, b=6, y0=1, t=1)
# -> y = 2.72933, y_steady = 3   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Solve y' + a y = b with a = 2, b = 6, y(0) = 1; find y(1).	SOLVED	y = 2.72933, y_steady = 3
Solve y' + a y = b with a = 0, b = 6, y(0) = 1; find y(1).	INVALID_INPUT	First-order linear ODE with constant coefficients (integrating factor): a must not be 0 (then y = y0 + b t directly).

First-order linear ODE with exponential forcing — `linear_first_order_exponential_forcing`

Solve y' + a $y = b$ e^{c t}, y(0) = y0, and give y(t) (including the resonant case c = -a).

$$y = \frac{b}{a+c}e^{ct} + \left(y_0 - \frac{b}{a+c}\right)e^{-at}$$

input	meaning	condition
`a`	coefficient of y	
`b`	forcing amplitude	
`c`	forcing exponent	
`y0`	initial value	
`t`	time	

output	meaning
`y`	value y(t)
`resonant`	$c = -a$ (secular term t e^{-a t})

- **Method:** integrating factor: $y = b/(a+c)$ e^{ct} + (y0 - b/(a+c)) e^{-at}; $c = -a$: (y0 + b t) e^{-at}
- **Assumes:** any real a, b, c
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
differential_equations_ai999x.solve('linear_first_order_exponential_forcing', a=1, b=2, c=0.5,
y0=0, t=2)
# -> y = 3.44393, resonant = False   (SOLVED)
```

```
differential_equations_ai999x.solve('linear_first_order_exponential_forcing', a=1, b=2, c=-1, y0=1,
t=2)
# -> y = 0.676676, resonant = True   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Solve y' + a y = b e^(c t) with a = 1, b = 2, c = 0.5, y(0) = 0; find y(2).	SOLVED	y = 3.44393, resonant = False
Solve y' + a y = b e^(c t) with a = 1, b = 2, y(0) = 0; find y(2).	NEEDS_INPUT	linear_first_order_exponential_forcing needs c (forcing exponent). Example: solve('linear_first_order_exponential_forcin

Mixing tank (brine): amount of salt at a time — `mixing_tank`

A well-stirred tank of volume V with inflow = outflow rate r at concentration c_in: amount A(t) of solute.

$$A(t) = c_{in}V + (A_0 - c_{in}V)e^{-rt/V}$$

input	meaning	condition
`V`	volume (constant, > 0)	
`r`	flow rate in = out (> 0)	
`c_in`	inflow concentration (>= 0)	
`A0`	initial amount of solute	
`t`	time	

output	meaning
`A`	amount A(t)
`A_limit`	long-run amount c_in V
`concentration`	concentration A(t)/V

- **Method:** A' = r c_in - (r/V) A: $A = c_i n$ V + (A0 - c_in V) e^{-r t/V}
- **Assumes:** $V > 0$; $r > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
differential_equations_ai999x.solve('mixing_tank', V=100, r=5, c_in=0.2, A0=0, t=10)
# -> A = 7.86939, A_limit = 20, concentration = 0.0786939   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
A brine mixing tank has V = 100, r = 5, c_in = 0.2, A0 = 0; find A(10).	SOLVED	A = 7.86939, A_limit = 20, concentration = 0.0786939
A brine mixing tank has V = 0, r = 5, c_in = 0.2, A0 = 0; find A(10).	INVALID_INPUT	Mixing tank (brine): amount of salt at a time: needs V > 0, r > 0, c_in >= 0, A0 >= 0.

Separable equation y' = a t^n y — `separable_power`

Solve the separable equation y' = a t^n y with y(0) = y0 and give y(t).

$$y(t) = y_0 \exp\left(\frac{a\, t^{n+1}}{n+1}\right)$$

input	meaning	condition
`a`	coefficient	
`n`	power of t (>= 0)	
`y0`	initial value	
`t`	time (>= 0)	

output	meaning
`y`	value y(t)

- **Method:** separate: $dy/y = a\, t^n$ dt, ln(y/y0) = a t^{$n + 1$}/$(n + 1)$
- **Assumes:** $n \geq 0$; $t \geq 0$

- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
differential_equations_ai999x.solve('separable_power', a=2, n=1, y0=3, t=1)
# -> y = 8.15485   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Solve the separable equation y' = a t^n y with a = 2, n = 1, y(0) = 3; find y(1).	SOLVED	y = 8.15485
Solve the separable equation y' = a t^n y with a = 2, n = -1, y(0) = 3; find y(1).	INVALID_INPUT	Separable equation y' = a t^n y: needs n >= 0 and t >= 0.

Equation y' = k y^2: solution and blow-up time — `quadratic_ode_blowup`

Solve y' = k y^2, y(0) = y0; give y(t) and the finite blow-up time 1/(k y0) when k $y_0 > 0$.

$$y(t) = \frac{y_0}{1 - k y_0 t}$$

input	meaning	condition
`k`	coefficient	
`y0`	initial value	
`t`	time (>= 0)	

output	meaning
`y`	value y(t)
`blowup_time`	time at which y becomes infinite (k y0 > 0)

- **Method:** separate: $-1/y = k\,t - 1/y_0$
- **Assumes:** t before the blow-up time
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
differential_equations_ai999x.solve('quadratic_ode_blowup', k=1, y0=0.5, t=1)
# -> y = 1, blowup_time = 2   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Solve y' = k y^2 with k = 1, y(0) = 0.5; find y(1).	SOLVED	y = 1, blowup_time = 2
Solve y' = k y^2 with k = 1, y(0) = 0.5; find y(3).	INVALID_INPUT	Equation y' = k y^2: solution and blow-up time: the solution blows up at t = 2, before the requested time.

Bernoulli equation y' + p y = q y^n — `bernoulli_equation`

Solve the Bernoulli equation with constant p, q by the substitution $v = y^{1-n}$; give y(t).

$$y^{1-n} = \frac{q}{p} + \left(y_0^{1-n} - \frac{q}{p}\right)e^{-(1-n)pt}$$

input	meaning	condition
`p`	coefficient of y	$p \neq 0$
`q`	coefficient of y^n	
`n`	exponent $(n \neq 1)$	
`y0`	initial value (> 0)	
`t`	time (>= 0)	

output	meaning
`y`	value y(t)

- **Method:** $v = y^{1-n}$: v' + $(1-n)$ p $v = (1-n)$ q, linear
- **Assumes:** $p \neq 0$; $n \neq 1$; $y_0 > 0$; solution stays positive
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
differential_equations_ai999x.solve('bernoulli_equation', p=1, q=1, n=2, y0=0.5, t=1)
# -> y = 0.268941   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Solve the Bernoulli equation y' + p y = q y^n with p = 1, q = 1, n = 2, y(0) = 0.5; find y(1).	SOLVED	y = 0.268941
Solve the Bernoulli equation y' + p y = q y^n with p = 1, q = 1, n = 1, y(0) = 0.5; find y(1).	INVALID_INPUT	Bernoulli equation y' + p y = q y^n: needs p != 0, n != 1, y0 > 0 and t >= 0.

Falling body with linear drag: velocity — `linear_drag_velocity`

Solve m v' = m g - c v with v(0) = v0: velocity at time t and terminal velocity m g / c.

$$v(t) = \frac{mg}{c} + \left(v_0 - \frac{mg}{c}\right)e^{-ct/m}$$

input	meaning	condition
`m`	mass (> 0)	
`c`	linear drag coefficient (> 0)	
`g`	gravitational acceleration	
`v0`	initial velocity (downward positive)	
`t`	time	

output	meaning
`v`	velocity v(t)
`v_terminal`	terminal velocity m g / c

- **Method:** linear first-order ODE: $v = mg/c + (v_0 - mg/c)$ e^{-ct/m}
- **Assumes:** $m > 0$; $c > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
differential_equations_ai999x.solve('linear_drag_velocity', m=2, c=0.5, g=9.81, v0=0, t=3)
# -> v = 20.7043, v_terminal = 39.24   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
A falling body with linear drag: m = 2, c = 0.5, g = 9.81, v0 = 0; find v(3).	SOLVED	v = 20.7043, v_terminal = 39.24
A falling body with linear drag: m = 2, c = 0.5, v0 = 0; find v(3).	NEEDS_INPUT	linear_drag_velocity needs g (gravitational acceleration). Example: solve('linear_drag_velocity', m=2, c=0.5, g=9.81, v0

Falling body with quadratic drag: velocity — `quadratic_drag_velocity`

Solve m v' = m g - c v^2 with v(0) = v0 >= 0: velocity at time t and terminal velocity sqrt(m g / c).

$$v(t) = v_t \tanh\left(\frac{gt}{v_t} + \operatorname{artanh}\frac{v_0}{v_t}\right),\ v_t = \sqrt{mg/c}$$

input	meaning	condition
`m`	mass (> 0)	
`c`	quadratic drag coefficient (> 0)	
`g`	gravitational acceleration (> 0)	
`v0`	initial downward velocity (>= 0)	
`t`	time (>= 0)	

output	meaning
`v`	velocity v(t)
`v_terminal`	terminal velocity

- **Method:** separable: $v = v_t$ tanh(g t / v_t + artanh(v0/v_t)) (coth form when v0 > v_t)
- **Assumes:** m, c, $g > 0$; $v_0 \geq 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
differential_equations_ai999x.solve('quadratic_drag_velocity', m=80, c=0.25, g=9.81, v0=0, t=5)
# -> v = 39.4514, v_terminal = 56.0286   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
A falling body with quadratic drag: m = 80, c = 0.25, g = 9.81, v0 = 0; find v(5).	SOLVED	v = 39.4514, v_terminal = 56.0286
A falling body with quadratic drag: m = 80, c = 0, g = 9.81, v0 = 0; find v(5).	INVALID_INPUT	Falling body with quadratic drag: velocity: needs m, c, g > 0, v0 >= 0 and t >= 0.

Draining tank (Torricelli's law): height and emptying time — `torricelli_draining`

Solve $dh/dt = -k$ sqrt(h), h(0) = h0: water height at time t and the time to empty.

$$h(t) = \left(\sqrt{h_0} - \tfrac{kt}{2}\right)^2,\ t_{empty} = \frac{2\sqrt{h_0}}{k}$$

input	meaning	condition
`h0`	initial height (> 0)	
`k`	drain constant (> 0)	
`t`	time (>= 0)	

output	meaning
`h`	height h(t) (0 once empty)
`t_empty`	time to empty 2 sqrt(h0)/k

- **Method:** separable: sqrt(h) = sqrt(h0) - k $t/2$
- **Assumes:** $h_0 > 0$; $k > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
differential_equations_ai999x.solve('torricelli_draining', h0=4, k=0.2, t=5)
# -> h = 2.25, t_empty = 20   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
A draining tank obeys Torricelli's law with h0 = 4, k = 0.2; find h(5) and the emptying time.	SOLVED	h = 2.25, t_empty = 20
A draining tank obeys Torricelli's law with h0 = -4, k = 0.2; find h(5).	INVALID_INPUT	Draining tank (Torricelli's law): height and emptying time: needs h0 > 0, k > 0 and t >= 0.

Gompertz growth: value at a time — `gompertz_growth`

Solve $dy/dt = r$ y ln(K/y), y(0) = y0 (tumour and population growth) and give y(t).

$$y(t) = K \exp\left(\ln \frac{y_0}{K}\, e^{-rt}\right)$$

input	meaning	condition
`y0`	initial value (> 0)	

input	meaning	condition
`r`	growth rate	
`K`	carrying capacity (> 0)	
`t`	time	

output	meaning
`y`	value y(t)

- **Method:** $u = ln$(y/K) satisfies u' = -r u
- **Assumes:** $y_0 > 0$; $K > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
differential_equations_ai999x.solve('gompertz_growth', y0=10, r=0.3, K=1000, t=5)
# -> y = 357.882   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Gompertz growth with y0 = 10, r = 0.3, K = 1000; find y(5).	SOLVED	y = 357.882
Gompertz growth with y0 = -10, r = 0.3, K = 1000; find y(5).	INVALID_INPUT	Gompertz growth: value at a time: needs y0 > 0 and K > 0.

Slope (direction) field value — `slope_field`

The slope $dy/dt = f$(t, y) of the direction field at a point, and its angle.

$$m = f(t, y),\ \theta = \arctan m$$

input	meaning	condition
`f`	right-hand side f(t, y)	
`t`	t-coordinate	
`y`	y-coordinate	

output	meaning
`slope`	slope f(t, y)
`angle_deg`	angle of the field line, degrees

- **Method:** evaluate f; angle = atan(slope)
- **Assumes:** f defined at the point
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
differential_equations_ai999x.solve('slope_field', f='t + y', t=1, y=2)
# -> slope = 3, angle_deg = 71.5651   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Slope field of dy/dt = t + y at t = 1, y = 2.	SOLVED	slope = 3, angle_deg = 71.5651
Slope field of dy/dt = log(y) at t = 1, y = -2.	INVALID_INPUT	Slope (direction) field value: the expression is not defined there (ValueError).

Euler's method — `euler_method`

Approximate the solution of y' = f(t, y), y(t0) = y0, after n steps of size h with Euler's method.

$$y_{k+1} = y_k + hf(t_k, y_k)$$

input	meaning	condition
`f`	right-hand side f(t, y) of y' = f(t, y)	
`t0`	initial time	
`y0`	initial value y(t0)	
`h`	step size (> 0)	
`n`	number of steps	

output	meaning
`y_n`	approximation of y($t_0 + n$ h)
`t_n`	final time $t_0 + n$ h

- **Method:** explicit Euler: y_{$k + 1$} = y_k + h f(t_k, y_k)
- **Assumes:** $h > 0$; $1 \le n \le 10000$; f defined along the steps
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
differential_equations_ai999x.solve('euler_method', f='t + y', t0=0, y0=1, h=0.1, n=2)
# -> y_n = 1.22, t_n = 0.2   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Use Euler's method for dy/dt = t + y, y(0) = 1, h = 0.1, n = 2.	SOLVED	y_n = 1.22, t_n = 0.2
Use Euler's method for dy/dt = t + y, y(0) = 1, h = 0.1.	NEEDS_INPUT	euler_method needs n (number of steps). Example: solve('euler_method', f='t + y', t0=0, y0=1, h=0.1, n=2).

Improved Euler (Heun) method — `improved_euler_method`

Approximate the solution of y' = f(t, y), y(t0) = y0, after n steps of size h with the improved Euler (Heun) method.

$$y_{k+1} = y_k + \frac{h}{2}[f(t_k, y_k) + f(t_{k+1}, y_k + hf(t_k, y_k))]$$

input	meaning	condition
`f`	right-hand side f(t, y) of y' = f(t, y)	
`t0`	initial time	
`y0`	initial value y(t0)	
`h`	step size (> 0)	
`n`	number of steps	

output	meaning
`y_n`	approximation of y($t_0 + n$ h)
`t_n`	final time $t_0 + n$ h

- **Method:** Heun predictor-corrector: average the slopes at both ends
- **Assumes:** $h > 0$; $1 \le n \le 10000$; f defined along the steps
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
differential_equations_ai999x.solve('improved_euler_method', f='t + y', t0=0, y0=1, h=0.1, n=2)
# -> y_n = 1.24205, t_n = 0.2   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Use the improved Euler method for dy/dt = t + y, y(0) = 1, h = 0.1, n = 2.	SOLVED	y_n = 1.24205, t_n = 0.2
Use the improved Euler method for dy/dt = t + y, y(0) = 1, h = 0.1.	NEEDS_INPUT	improved_euler_method needs n (number of steps). Example: solve('improved_euler_method', f='t + y', t0=0, y0=1, h=0.1, n

Explicit midpoint method (RK2) — `midpoint_method`

Approximate the solution of y' = f(t, y), y(t0) = y0, after n steps of size h with the explicit midpoint method.

$$y_{k+1} = y_k + h f\left(t_k + \tfrac{h}{2}, y_k + \tfrac{h}{2} f(t_k, y_k)\right)$$

input	meaning	condition
`f`	right-hand side f(t, y) of y' = f(t, y)	
`t0`	initial time	
`y0`	initial value y(t0)	
`h`	step size (> 0)	
`n`	number of steps	

output	meaning
`y_n`	approximation of y($t_0 + n$ h)
`t_n`	final time $t_0 + n$ h

- **Method:** second-order Runge-Kutta, slope at the midpoint

- **Assumes:** $h > 0$; $1 \leq n \leq 10000$; f defined along the steps
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
differential_equations_ai999x.solve('midpoint_method', f='t + y', t0=0, y0=1, h=0.1, n=2)
# -> y_n = 1.24205, t_n = 0.2   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Apply the midpoint method to dy/dt = t + y, y(0) = 1, h = 0.1, n = 2.	SOLVED	y_n = 1.24205, t_n = 0.2
Apply the midpoint method to dy/dt = t + y, y(0) = 1, h = 0.1.	NEEDS_INPUT	midpoint_method needs n (number of steps). Example: solve('midpoint_method', f='t + y', t0=0, y0=1, h=0.1, n=2).

Classical Runge-Kutta method (RK4) — `runge_kutta_4`

Approximate the solution of y' = f(t, y), y(t0) = y0, after n steps of size h with the classical fourth-order Runge-Kutta method.

$$y_{k+1} = y_k + \tfrac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4)$$

input	meaning	condition
`f`	right-hand side f(t, y) of y' = f(t, y)	
`t0`	initial time	
`y0`	initial value y(t0)	
`h`	step size (> 0)	
`n`	number of steps	

output	meaning
`y_n`	approximation of y($t_0 + n$ h)
`t_n`	final time $t_0 + n$ h

- **Method:** classical RK4 with four slope evaluations per step
- **Assumes:** $h > 0$; $1 \leq n \leq 10000$; f defined along the steps
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
differential_equations_ai999x.solve('runge_kutta_4', f='t + y', t0=0, y0=1, h=0.1, n=2)
# -> y_n = 1.24281, t_n = 0.2   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Use RK4 for dy/dt = t + y, y(0) = 1, h = 0.1, n = 2.	SOLVED	y_n = 1.24281, t_n = 0.2

question	outcome	result
Use RK4 for dy/dt = t + y, y(0) = 1, h = 0.1.	NEEDS_INPUT	runge_kutta_4 needs n (number of steps). Example: solve('runge_kutta_4', f='t + y', t0=0, y0=1, h=0.1, n=2).

Backward (implicit) Euler method — `backward_euler_method`

Approximate y' = f(t, y), y(t0) = y0 after n implicit Euler steps: y_{$k+1$} = y_k + h f(t_{$k+1$}, y_{$k+1$}).

$$y_{k+1} = y_k + h f(t_{k+1}, y_{k+1})$$

input	meaning	condition
`f`	right-hand side f(t, y) of y' = f(t, y)	
`t0`	initial time	
`y0`	initial value y(t0)	
`h`	step size (> 0)	
`n`	number of steps	

output	meaning
`y_n`	approximation of y($t_0 + n$ h)
`t_n`	final time $t_0 + n$ h

- **Method:** each step solved by Newton's method from the explicit Euler predictor
- **Assumes:** $h > 0$; h*|df/dy| < 1 at each step (unique step)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
differential_equations_ai999x.solve('backward_euler_method', f='-2*y + t', t0=0, y0=1, h=0.1, n=2)
# -> y_n = 0.718056, t_n = 0.2   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Use the backward Euler method for dy/dt = -2*y + t, y(0) = 1, h = 0.1, n = 2.	SOLVED	y_n = 0.718056, t_n = 0.2
Use the backward Euler method for dy/dt = -2*y + t, y(0) = 1, h = 0.1.	NEEDS_INPUT	backward_euler_method needs n (number of steps). Example: solve('backward_euler_method', f='-2*y + t', t0=0, y0=1, h=0.1

Euler's method error for exponential growth — `euler_error_exponential`

For y' = k y, y(0) = y0: the Euler approximation after n steps of size h, the exact value and the global error.

$$y_n = y_0(1 + hk)^n,\ y(t_n) = y_0 e^{k t_n}$$

input	meaning	condition
`k`	rate constant	

input	meaning	condition
`y0`	initial value	
`h`	step size (> 0)	
`n`	number of steps	

output	meaning
`y_euler`	Euler value y0 (1 + h k)^n
`y_exact`	exact value y0 e^{k n h}
`error`	global error exact - Euler
`relative_error`	error / exact

- **Method:** Euler on a linear equation multiplies by $(1 + hk)$ per step
- **Assumes:** $h > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
differential_equations_ai999x.solve('euler_error_exponential', k=1, y0=1, h=0.1, n=10)
# -> y_euler = 2.59374, y_exact = 2.71828, error = 0.124539, relative_error = 0.0458155   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the global error of Euler's method for y' = k y with k = 1, y0 = 1, h = 0.1, n = 10.	SOLVED	y_euler = 2.59374, y_exact = 2.71828, error = 0.124539, relative_error = 0.0458155
Find the global error of Euler's method for y' = k y with k = 1, y0 = 1, h = -0.1, n = 10.	INVALID_INPUT	Euler's method error for exponential growth: needs h > 0 and 1 <= n <= 10000.

RK4 for a system of two first-order equations — `runge_kutta_4_system`

Approximate x' = f(t, x, y), y' = g(t, x, y) with x(t0) = x0, y(t0) = y0 after n RK4 steps of size h.

$$\mathbf{u}_{k+1} = \mathbf{u}_k + \tfrac{h}{6}(\mathbf{k}_1 + 2\mathbf{k}_2 + 2\mathbf{k}_3 + \mathbf{k}_4)$$

input	meaning	condition
`f`	x' = f(t, x, y)	
`g`	y' = g(t, x, y)	
`t0`	initial time	
`x0`	initial x	
`y0`	initial y	
`h`	step size (> 0)	
`n`	number of steps	

output	meaning
`x_n`	approximation of x($t_0 + n$ h)
`y_n`	approximation of y($t_0 + n$ h)

output	meaning
`t_n`	final time

- **Method:** classical RK4 applied to the vector (x, y)
- **Assumes:** $h > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
differential_equations_ai999x.solve('runge_kutta_4_system', f='y', g='-x', t0=0, x0=1, y0=0, h=0.1,
n=10)
# -> x_n = 0.540303, y_n = -0.84147, t_n = 1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Use RK4 on the system dx/dt = y, dy/dt = -x with x(0) = 1, y(0) = 0, t0 = 0, h = 0.1, n = 10.	SOLVED	x_n = 0.540303, y_n = -0.84147, t_n = 1
Use RK4 on the system dx/dt = y, dy/dt = -x with x(0) = 1, y(0) = 0, t0 = 0, h = 0.1.	NEEDS_INPUT	runge_kutta_4_system needs n (number of steps). Example: solve('runge_kutta_4_system', f='y', g='-x', t0=0, x0=1, y0=0,

Second-order constant-coefficient ODE: characteristic roots — `characteristic_roots`

Roots of the characteristic equation of a y'' + b y' + c $y = 0$, the form of the general solution and the damping case.

$$ar^2 + br + c = 0, \quad r = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

input	meaning	condition
`a`	coefficient of y''	$a \neq 0$
`b`	coefficient of y'	
`c`	coefficient of y	

output	meaning
`r1`	first root (real ascending; complex: positive imaginary part first)
`r2`	second root
`discriminant`	$b^2 - 4ac$
`roots_type`	distinct real / repeated / complex
`damping`	overdamped / critically damped / underdamped / undamped
`solution_form`	form of the general solution

- **Method:** a $r^2 + b\,r + c = 0$ (stable quadratic formula); case from the exact sign of $b^2 - 4ac$
- **Assumes:** $a \neq 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
differential_equations_ai999x.solve('characteristic_roots', a=1, b=3, c=2)
# -> r1 = -2, r2 = -1, discriminant = 1, roots_type = two distinct real roots, damping =
overdamped, solution_form = y = C1 e^(r1 t) + C2 e^(r2 t)   (SOLVED)
```

```
differential_equations_ai999x.solve('characteristic_roots', a=1, b=2, c=5)
# -> r1 = -1 + 2i, r2 = -1 - 2i, discriminant = -16, roots_type = complex conjugate roots, damping
= underdamped, solution_form = y = e^(alpha t) (C1 cos(beta t) + C2 sin(beta t)), alpha = Re r1,
beta = Im r1   (SOLVED)
```

```
differential_equations_ai999x.solve('characteristic_roots', a=1, b=4, c=4)
# -> r1 = -2, r2 = -2, discriminant = 0, roots_type = one repeated real root, damping = critically
damped, solution_form = y = (C1 + C2 t) e^(r t)   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the characteristic roots of y'' + 3y' + 2y = 0.	SOLVED	r1 = -2, r2 = -1, discriminant = 1, roots_type = two distinct real roots, damping = overdamped, solution_form = y = C1 e^(r1 t) + C2 e^(r2 t)
Characteristic equation of a y'' + b y' + c y = 0 with a = 1, b = 2, c = 5.	SOLVED	r1 = -1 + 2i, r2 = -1 - 2i, discriminant = -16, roots_type = complex conjugate roots, damping = underdamped, solution_form = y = e^(alpha t) (C1 cos(beta t) + C2 sin(beta t)), alpha = Re r1, beta = Im r1
Find the characteristic roots of y'' + 9y = 0.	SOLVED	r1 = 0 + 3i, r2 = 0 - 3i, discriminant = -36, roots_type = complex conjugate roots, damping = undamped, solution_form = y = e^(alpha t) (C1 cos(beta t) + C2 sin(beta t)), alpha = Re r1, beta = Im r1
Characteristic equation of a y'' + b y' + c y = 0 with a = 0, b = 2, c = 5.	INVALID_INPUT	Second-order constant-coefficient ODE: characteristic roots: a must not be 0 (the equation would be first order).

Second-order homogeneous linear ODE: initial value problem — `second_order_homogeneous_ivp`

Solve a y'' + b y' + c $y = 0$ with y(0) = y0, y'$(0) = v_0$ and give y(t) (overdamped, critical or underdamped).

$$y(t) = e^{mt}\left[y_0\cosh(\delta t) + (v_0 - m y_0)\frac{\sinh(\delta t)}{\delta}\right],\ m = -\frac{b}{2a},\ \delta^2 = m^2 - \frac{c}{a}$$

input	meaning	condition
`a`	coefficient of y''	$a \neq 0$
`b`	coefficient of y'	
`c`	coefficient of y	
`y0`	initial value y(0)	
`v0`	initial slope y'(0)	
`t`	time	

output	meaning
`y`	value y(t)
`damping`	damping case

- **Method:** $y = e$^{mt}[y0 C(t) + ($v_0 - m$ y0) S(t)], $m = -b/(2a)$, C, S = cosh/sinh, $1/t$ or cos/sin by the sign of the discriminant
- **Assumes:** $a \neq 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
differential_equations_ai999x.solve('second_order_homogeneous_ivp', a=1, b=3, c=2, y0=1, v0=0, t=1)
# -> y = 0.600424, damping = overdamped   (SOLVED)
```

```
differential_equations_ai999x.solve('second_order_homogeneous_ivp', a=1, b=0, c=4, y0=1, v0=0, t=1)
# -> y = -0.416147, damping = undamped   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Solve y'' + 3y' + 2y = 0 with y(0) = 1, y'(0) = 0; find y(1).	SOLVED	y = 0.600424, damping = overdamped
Solve y'' + 4y = 0 with y(0) = 1, y'(0) = 0; find y(1).	SOLVED	y = -0.416147, damping = undamped
Solve y'' + 3y' + 2y = 0 with y(0) = 1; find y(1).	NEEDS_INPUT	second_order_homogeneous_ivp needs v0 (initial slope y'(0)). Example: solve('second_order_homogeneous_ivp', a=1, b=3, c=

Undetermined coefficients: exponential forcing — `undetermined_coefficients_exponential`

Solve a y'' + b y' + c $y = F$ e^{s t}, y(0) = y0, y'$(0) = v_0$: the particular coefficient (with resonance) and y(t).

$$y_p = \frac{F}{as^2 + bs + c}\, e^{st}$$

input	meaning	condition
`a`	coefficient of y''	$a \neq 0$
`b`	coefficient of y'	
`c`	coefficient of y	
`F`	forcing amplitude	
`s`	forcing exponent	
`y0`	initial value	
`v0`	initial slope	
`t`	time	

output	meaning
`y`	value y(t)
`A`	coefficient of the particular solution A t^m e^{st}
`resonance_multiplicity`	m: 0, or $1/2$ when s is a simple/double characteristic root

- **Method:** $y_p = A\, t^m$ e^{st}, $A = F/P$(s), F/P'(s) or $F/(2a)$; plus the homogeneous solution fitted to the initial values

- **Assumes:** $a \neq 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
differential_equations_ai999x.solve('undetermined_coefficients_exponential', a=1, b=3, c=2, F=4,
s=3, y0=0, v0=0, t=1)
# -> y = 3.7575, A = 0.2, resonance_multiplicity = 0   (SOLVED)
```

```
differential_equations_ai999x.solve('undetermined_coefficients_exponential', a=1, b=3, c=2, F=1,
s=-1, y0=0, v0=0, t=2)
# -> y = 0.153651, A = 1, resonance_multiplicity = 1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Solve y'' + 3y' + 2y = 4e^(3t) with y(0) = 0, y'(0) = 0; find y(1).	SOLVED	y = 3.7575, A = 0.2, resonance_multiplicity = 0
Solve y'' + 3y' + 2y = 4e^(3t) with y(0) = 0; find y(1).	NEEDS_INPUT	undetermined_coefficients_exponential needs v0 (initial slope). Example: solve('undetermined_coefficients_exponential',

Undetermined coefficients: polynomial forcing — `undetermined_coefficients_polynomial`

Solve a y'' + b y' + c $y = p_0 + p_1$ t + ... (degree <= 4), y(0) = y0, y'$(0) = v_0$: particular polynomial and y(t).

$$y_p = \sum_k q_k t^k, \quad c\,q_k + b(k+1)q_{k+1} + a(k+2)(k+1)q_{k+2} = p_k$$

input	meaning	condition
`a`	coefficient of y''	$a \neq 0$
`b`	coefficient of y'	
`c`	coefficient of y	$c \neq 0$
`p`	forcing coefficients p0, p1, ... (ascending powers of t)	
`y0`	initial value	
`v0`	initial slope	
`t`	time	

output	meaning
`y`	value y(t)
`particular`	coefficients q0, q1, ... of the particular polynomial

- **Method:** match powers of t from the top: c q_k + b $(k+1)$ q_{$k+1$} + a $(k+2)(k+1)$ q_{$k+2$} = p_k
- **Assumes:** $a \neq 0$; $c \neq 0$; degree <= 4
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
differential_equations_ai999x.solve('undetermined_coefficients_polynomial', a=1, b=0, c=4, p='0, 8', y0=1, v0=0, t=0.5)
# -> y = 0.698831, particular = [0.0, 2.0]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Undetermined coefficients with polynomial forcing: a = 1, b = 0, c = 4, p = [0, 8], y0 = 1, v0 = 0, t = 0.5.	SOLVED	y = 0.698831, particular = [0.0, 2.0]
Solve y'' + 3y' = 4t^2 + 1 with y(0) = 0, y'(0) = 0; find y(1).	INVALID_INPUT	Undetermined coefficients: polynomial forcing: c must not be 0 (integrate once: the equation is first order in y').

Sinusoidally forced linear oscillator: initial value problem — `forced_oscillator_ivp`

Solve a y'' + b y' + c $y = F_0$ cos(w t), y(0) = y0, y'$(0) = v_0$ (including pure resonance) and give y(t).

$$y_p = \mathrm{Re}\,\frac{F_0 e^{i\omega t}}{c - a\omega^2 + ib\omega}$$

input	meaning	condition
`a`	coefficient of y''	$a \neq 0$
`b`	coefficient of y'	
`c`	coefficient of y	
`F0`	forcing amplitude	
`w`	forcing angular frequency (> 0)	
`y0`	initial value	
`v0`	initial slope	
`t`	time	

output	meaning
`y`	value y(t)
`resonant`	undamped forcing at the natural frequency

- **Method:** complex amplitude Z = F0/(c - a $w^2 + i$ b w); resonance: $y_p = F_0/(2aw)$ t sin(wt); plus homogeneous part
- **Assumes:** $a \neq 0$; $w > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
differential_equations_ai999x.solve('forced_oscillator_ivp', a=1, b=0, c=4, F0=1, w=2, y0=0, v0=0, t=3)
# -> y = -0.209562, resonant = True   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
A forced oscillator a y'' + b y' + c y = F0 cos(w t) with a = 1, b = 0, c = 4, F0 = 1, w = 2, y0 = 0, v0 = 0, t = 3.	SOLVED	y = -0.209562, resonant = True
Solve y'' + 2y' + 5y = 3cos(2t) with y(0) = 0; find y(1).	NEEDS_INPUT	forced_oscillator_ivp needs v0 (initial slope). Example: solve('forced_oscillator_ivp', a=1, b=0, c=4, F0=1, w=2, y0=0,

Forced damped oscillator: steady-state amplitude and phase — `steady_state_amplitude`

Steady-state response of a y'' + b y' + c $y = F_0$ cos(w t): amplitude, phase lag and gain.

$$A = \frac{F_0}{\sqrt{(c - a\omega^2)^2 + (b\omega)^2}},\ \tan\delta = \frac{b\omega}{c - a\omega^2}$$

input	meaning	condition
`a`	mass-like coefficient (> 0)	
`b`	damping (> 0)	
`c`	stiffness (> 0)	
`F0`	forcing amplitude	
`w`	forcing angular frequency (> 0)	

output	meaning
`amplitude`	steady-state amplitude
`phase_lag`	phase lag in radians (0 to pi)
`gain`	amplitude per unit force

- **Method:** |F0| / sqrt((c - a $w^2)^2 + (b$ w)^2), lag atan2(b w, c - a w^2)
- **Assumes:** a, b, $c > 0$; $w > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
differential_equations_ai999x.solve('steady_state_amplitude', a=1, b=0.5, c=4, F0=2, w=1.5)
# -> amplitude = 1.05045, phase_lag = 0.404892, gain = 0.525226   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the steady-state amplitude of y'' + b y' + c y = F0 cos(w t) with a = 1, b = 0.5, c = 4, F0 = 2, w = 1.5.	SOLVED	amplitude = 1.05045, phase_lag = 0.404892, gain = 0.525226
Find the steady-state amplitude with a = 1, b = 0, c = 4, F0 = 2, w = 1.5.	INVALID_INPUT	Forced damped oscillator: steady-state amplitude and phase: needs a, b, c > 0 (damped oscillator) and w > 0.

Damped oscillator: resonance frequency and peak amplitude — `practical_resonance`

For m y'' + c y' + k $y = F_0$ cos(w t): the forcing frequency of maximum steady-state amplitude and that amplitude.

$$\omega_r = \sqrt{\frac{k}{m} - \frac{c^2}{2m^2}}, \quad A_{\max} = \frac{F_0}{c\sqrt{k/m - c^2/(4m^2)}}$$

input	meaning	condition
`m`	mass (> 0)	
`c`	damping coefficient (> 0)	
`k`	spring constant (> 0)	
`F0`	forcing amplitude	

output	meaning
`omega_r`	resonance (peak) frequency, 0 when there is no peak
`amplitude_max`	peak amplitude
`has_peak`	$c^2 < 2$ m k

- **Method:** minimise (k - m $w^2)^2 + (c$ w)^2 over w: $w_r^2 = k/m - c^2/(2m^2)$
- **Assumes:** m, c, $k > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
differential_equations_ai999x.solve('practical_resonance', m=1, c=0.4, k=4, F0=2)
# -> omega_r = 1.9799, amplitude_max = 2.51259, has_peak = True   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the practical resonance frequency for m = 1, c = 0.4, k = 4, F0 = 2.	SOLVED	omega_r = 1.9799, amplitude_max = 2.51259, has_peak = True
Find the practical resonance frequency for m = 1, c = 0.4, k = 4.	NEEDS_INPUT	practical_resonance needs F0 (forcing amplitude). Example: solve('practical_resonance', m=1, c=0.4, k=4, F0=2).

Undamped resonance: growing response — `undamped_resonance`

m y'' + k $y = F_0$ cos(w0 t) forced exactly at the natural frequency, starting from rest: y(t) and its linearly growing envelope.

$$y(t) = \frac{F_0}{2m\omega_0}\, t \sin(\omega_0 t), \quad \omega_0 = \sqrt{k/m}$$

input	meaning	condition
`m`	mass (> 0)	
`k`	spring constant (> 0)	

input	meaning	condition
`F0`	forcing amplitude	
`t`	time	

output	meaning
`y`	displacement y(t)
`envelope`	amplitude envelope F0 t / (2 m w0)
`omega0`	natural frequency

- **Method:** particular solution t sin(w0 t)/(2 m w0) (secular term)
- **Assumes:** m, $k > 0$; y(0) = y'$(0) = 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
differential_equations_ai999x.solve('undamped_resonance', m=1, k=4, F0=2, t=3)
# -> y = -0.419123, envelope = 1.5, omega0 = 2   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Undamped resonance with m = 1, k = 4, F0 = 2: find y(3).	SOLVED	y = -0.419123, envelope = 1.5, omega0 = 2
Undamped resonance with m = 1, k = -4, F0 = 2: find y(3).	INVALID_INPUT	Undamped resonance: growing response: needs m > 0 and k > 0.

Impulse response of a second-order ODE — `ode_impulse_response`

The response h(t) of a y'' + b y' + c $y = \delta$(t) from rest (the inverse Laplace transform of 1/(a $s^2 + b$ s + c)).

$$h(t) = \mathcal{L}^{-1}\left\{\frac{1}{as^2 + bs + c}\right\}$$

input	meaning	condition
`a`	coefficient of y''	$a \neq 0$
`b`	coefficient of y'	
`c`	coefficient of y	
`t`	time (>= 0)	

output	meaning
`h`	impulse response h(t)

- **Method:** the impulse gives y(0+) = 0, y'(0+) = $1/a$; then the homogeneous solution
- **Assumes:** $a \neq 0$; $t \geq 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
differential_equations_ai999x.solve('ode_impulse_response', a=1, b=2, c=5, t=1)
# -> h = 0.167256   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the impulse response of y'' + 2y' + 5y = delta(t) at t = 1.	SOLVED	h = 0.167256
Find the impulse response of y'' + 2y' + 5y = delta(t).	NEEDS_INPUT	ode_impulse_response needs t (time (>= 0)). Example: solve('ode_impulse_response', a=1, b=2, c=5, t=1).

Unit-step response of a second-order ODE — `ode_step_response`

The response of a y'' + b y' + c $y = u$(t) (unit step) from rest, and its final value $1/c$ when stable.

$$y(t) = \mathcal{L}^{-1}\left\{\frac{1}{s(as^2+bs+c)}\right\}$$

input	meaning	condition
`a`	coefficient of y''	$a \neq 0$
`b`	coefficient of y'	
`c`	coefficient of y	$c \neq 0$
`t`	time (>= 0)	

output	meaning
`y`	step response y(t)
`final_value`	limit $1/c$ (stable systems)

- **Method:** Y(s) $= 1/($$s$(as^2 + bs + c)): constant $1/c$ plus the homogeneous solution with y(0) = -1/c, y'$(0) = 0$
- **Assumes:** a, $c \neq 0$; $t \geq 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
differential_equations_ai999x.solve('ode_step_response', a=1, b=2, c=5, t=1)
# -> y = 0.197167, final_value = 0.2   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the unit step response of y'' + 2y' + 5y = u(t) at t = 1.	SOLVED	y = 0.197167, final_value = 0.2
Find the unit step response of y'' + 2y' = u(t) at t = 1.	INVALID_INPUT	Unit-step response of a second-order ODE: needs a != 0, c != 0 and t >= 0.

Mass-spring-damper: natural frequency, damping ratio, damped frequency — `damped_oscillator_parameters`

For m x'' + c x' + k $x = 0$: natural frequency, damping ratio, damping case and (underdamped) damped frequency and period.

$$\omega_0 = \sqrt{k/m},\ \zeta = \frac{c}{2\sqrt{mk}},\ \omega_d = \omega_0\sqrt{1-\zeta^2}$$

input	meaning	condition
`m`	mass (> 0)	
`c`	damping coefficient (>= 0)	
`k`	spring constant (> 0)	

output	meaning
`omega0`	natural frequency sqrt(k/m)
`zeta`	damping ratio c/(2 sqrt(mk))
`damping`	damping case
`omega_d`	damped frequency (underdamped)
`period_d`	damped period (underdamped)

- **Method:** $w_0 = \sqrt{\frac{k}{m}}$, zeta = c/(2 sqrt(mk)), $w_d = w_0$ sqrt(1 - zeta^2)
- **Assumes:** $m > 0$; $c \geq 0$; $k > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
differential_equations_ai999x.solve('damped_oscillator_parameters', m=2, c=1, k=8)
# -> omega0 = 2, zeta = 0.125, damping = underdamped, omega_d = 1.98431, period_d = 3.16643
(SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the damping ratio of a mass-spring-damper with m = 2, c = 1, k = 8.	SOLVED	omega0 = 2, zeta = 0.125, damping = underdamped, omega_d = 1.98431, period_d = 3.16643
Find the damping ratio of a mass-spring-damper with m = 2, c = 1.	NEEDS_INPUT	damped_oscillator_parameters needs k (spring constant (> 0)). Example: solve('damped_oscillator_parameters', m=2, c=1, k

Simple harmonic motion: position, amplitude, period — `simple_harmonic_motion`

Solve m x'' + k $x = 0$ with x(0) = x0, x'$(0) = v_0$: position at t, amplitude, angular frequency and period.

$$x(t) = x_0\cos\omega t + \frac{v_0}{\omega}\sin\omega t,\ A = \sqrt{x_0^2 + (v_0/\omega)^2}$$

input	meaning	condition
`m`	mass (> 0)	
`k`	spring constant (> 0)	
`x0`	initial position	
`v0`	initial velocity	
`t`	time	

output	meaning
`x`	position x(t)
`amplitude`	amplitude
`omega`	angular frequency
`period`	period

- **Method:** $x = x_0$ cos(wt) + (v0/w) sin(wt), $w = \sqrt{\frac{k}{m}}$
- **Assumes:** m, $k > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
differential_equations_ai999x.solve('simple_harmonic_motion', m=0.5, k=8, x0=0.1, v0=0, t=1)
# -> x = -0.0653644, amplitude = 0.1, omega = 4, period = 1.5708   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Simple harmonic motion with m = 0.5, k = 8, x(0) = 0.1, x'(0) = 0; find x(1).	SOLVED	x = -0.0653644, amplitude = 0.1, omega = 4, period = 1.5708
Simple harmonic motion with m = 0.5, k = 8, x(0) = 0.1; find x(1).	NEEDS_INPUT	simple_harmonic_motion needs v0 (initial velocity). Example: solve('simple_harmonic_motion', m=0.5, k=8, x0=0.1, v0=0, t

Cauchy-Euler (equidimensional) equation: initial value problem — `cauchy_euler_ivp`

Solve a t^2 y'' + b t y' + c $y = 0$ $(t > 0)$ with y(1) = y1, y'$(1) = v_1$ and give y(t) and the indicial roots.

$$a\,m(m-1) + b\,m + c = 0, \quad y = t^m$$

input	meaning	condition
`a`	coefficient of t^2 y''	$a \neq 0$
`b`	coefficient of t y'	
`c`	coefficient of y	
`y1`	value y(1)	
`v1`	slope y'(1)	
`t`	time (> 0)	

output	meaning
`y`	value y(t)
`m1`	first indicial root
`m2`	second indicial root

- **Method:** $t = e^x$ turns it into a Y'' + (b - a) Y' + c $Y = 0$; indicial a m(m-1) + b $m + c = 0$
- **Assumes:** $a \neq 0$; $t > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
differential_equations_ai999x.solve('cauchy_euler_ivp', a=1, b=2, c=-6, y1=1, v1=0, t=2)
# -> y = 2.45, m1 = -3, m2 = 2   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Solve the Cauchy-Euler equation t^2 y'' + 2t y' - 6y = 0 with y(1) = 1, y'(1) = 0; find y(2).	SOLVED	y = 2.45, m1 = -3, m2 = 2
Solve the Cauchy-Euler equation t^2 y'' + 2t y' - 6y = 0 with y(1) = 1, y'(1) = 0; find y(-2).	INVALID_INPUT	Cauchy-Euler (equidimensional) equation: initial value problem: needs a != 0 and t > 0.

Linear two-point boundary value problem (constant coefficients) — `boundary_value_problem`

Solve y'' + p y' + q $y = r$ on [xa, xb] with y(xa) = ya, y(xb) = yb; give y(x) and the slope at xa.

$$y'' + py' + qy = r,\ y(x_a) = y_a,\ y(x_b) = y_b$$

input	meaning	condition
`p`	coefficient of y'	
`q`	coefficient of y	
`r`	constant right-hand side	
`xa`	left end	
`xb`	right end (> xa)	
`ya`	value at xa	
`yb`	value at xb	
`x`	point in [xa, xb]	

output	meaning
`y`	value y(x)
`slope_a`	slope y'(xa)

- **Method:** particular + homogeneous basis; boundary conditions fix the two constants
- **Assumes:** $xa < xb$; unique solution (not an eigenvalue case)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
differential_equations_ai999x.solve('boundary_value_problem', p=0, q=-1, r=0, xa=0, xb=1, ya=0,
yb=1, x=0.5)
# -> y = 0.443409, slope_a = 0.850918   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Solve the boundary value problem y'' + p y' + q y = r with p = 0, q = -1, r = 0, xa = 0, xb = 1, ya = 0, yb = 1, x = 0.5.	SOLVED	y = 0.443409, slope_a = 0.850918
Solve the boundary value problem y'' + p y' + q y = r with p = 0, q = 9.869604401089358, r = 0, xa = 0, xb = 1, ya = 0, yb = 1, x = 0.5.	INVALID_INPUT	Linear two-point boundary value problem (constant coefficients): the boundary value problem has no unique solution (an e

Laplace transform of a function at a point — `laplace_transform`

F(s) = integral from 0 to infinity of e^{-st} f(t) dt for sums of t^n e^{at} sin/cos(bt) (also sinh, cosh), and the abscissa of convergence.

$$\mathcal{L}\{t^n e^{zt}\}(s) = \frac{n!}{(s-z)^{n+1}}$$

input	meaning	condition
`f`	function f(t)	
`s`	transform variable (s > abscissa)	

output	meaning
`F`	transform value F(s)
`sigma`	abscissa of convergence (F exists for s > sigma)

- **Method:** table: L{t^n e^{zt}} = n!/(s - z)^{$n+1$}, with sin and cos written through complex exponentials
- **Assumes:** f a sum of t^n e^{at} sin/cos(bt) terms; $s > \sigma$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
differential_equations_ai999x.solve('laplace_transform', f='t^2*exp(3*t)', s=5)
# -> F = 0.25, sigma = 3   (SOLVED)
```

```
differential_equations_ai999x.solve('laplace_transform', f='sin(2t)', s=1)
# -> F = 0.4, sigma = 0   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the Laplace transform of t^2 e^(3t) at s = 5.	SOLVED	F = 0.25, sigma = 3
Laplace transform of sin(2t) at s = 1.	SOLVED	F = 0.4, sigma = 0

question	outcome	result
Laplace transform of exp(3t) at s = 2.	INVALID_INPUT	Laplace transform of a function at a point: the transform exists only for s > 3.
Laplace transform of log(t) at s = 2.	INVALID_INPUT	Laplace transform of a function at a point: supported: sums of c t^n e^{a t} sin/cos/sinh/cosh(b t).

Laplace transform of a delayed function u(t - c) f(t - c) — `laplace_time_shift`

Second shifting theorem: the transform of u(t - c) f(t - c) is e^{-cs} F(s).

$$\mathcal{L}\{u(t-c)f(t-c)\} = e^{-cs}F(s)$$

input	meaning	condition
`f`	function f(t)	
`c`	delay (>= 0)	
`s`	transform variable	

output	meaning
`F`	transform of the delayed function
`G`	transform of f itself

- **Method:** e^{-cs} times the table transform of f
- **Assumes:** $c \geq 0$; s > abscissa of f
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
differential_equations_ai999x.solve('laplace_time_shift', f='t', c=2, s=1)
# -> F = 0.135335, G = 1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the Laplace transform of the delayed function u(t - c) f(t - c) with f(t) = t, c = 2, s = 1.	SOLVED	F = 0.135335, G = 1
Find the Laplace transform of the delayed function u(t - c) f(t - c) with f(t) = t, c = -2, s = 1.	INVALID_INPUT	Laplace transform of a delayed function u(t - c) f(t - c): the delay c must be >= 0.

Inverse Laplace transform of a rational function — `inverse_laplace_transform`

f(t) for a strictly proper rational F(s) (any poles, repeated or complex), by residues / partial fractions.

$$f(t) = \sum_{p} \mathrm{Res}_{s=p}\ F(s)e^{st}$$

input	meaning	condition
`F`	F(s), a rational function of s	
`t`	time (>= 0)	

output	meaning
`f`	value f(t)
`poles`	poles of F(s) (canonical order)

- **Method:** f(t) = sum over poles p (order m) of the residue of F(s) e^{st}: e^{pt} sum_k g_k t^{$m-1-k$}/$(m-1-k)$!
- **Assumes:** F strictly proper with rational coefficients; denominator degree <= 10
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
differential_equations_ai999x.solve('inverse_laplace_transform', F='(s + 3)/(s^2 + 3s + 2)', t=1)
# -> f = 0.600424, poles = [-2.0, -1.0]   (SOLVED)
```

```
differential_equations_ai999x.solve('inverse_laplace_transform', F='1/(s^2 + 4)', t=2)
# -> f = -0.378401, poles = [2j, -2j]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the inverse Laplace transform of (s + 3)/(s^2 + 3s + 2) at t = 1.	SOLVED	f = 0.600424, poles = [-2.0, -1.0]
Inverse Laplace transform of 1/(s^2 + 4) at t = 2.	SOLVED	f = -0.378401, poles = [2j, -2j]
Inverse Laplace transform of 1/(s + 1)^2 at t = 1.	SOLVED	f = 0.367879, poles = [-1.0]
Inverse Laplace transform of s^2/(s + 1) at t = 1.	INVALID_INPUT	Inverse Laplace transform of a rational function: F(s) must be strictly proper (degree of numerator < degree of denomina

Partial fractions of a rational function (simple poles) — `partial_fraction_residues`

F(s) = sum r_i/(s - p_i): the poles and residues of a strictly proper rational function with simple poles.

$$F(s)=\sum_i \frac{r_i}{s-p_i}, \quad r_i=\frac{P(p_i)}{Q'(p_i)}$$

input	meaning	condition
`F`	F(s), a rational function of s	

output	meaning
`poles`	poles p_i (real ascending, then complex)
`residues`	residues r_i in the same order
`count`	number of poles

- **Method:** $r_i = P$(p_i)/Q'(p_i) (cover-up rule)
- **Assumes:** strictly proper; simple (distinct) poles
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
differential_equations_ai999x.solve('partial_fraction_residues', F='(s + 3)/(s^2 + 3s + 2)')
# -> poles = [-2.0, -1.0], residues = [-1.0, 2.0], count = 2   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the partial fractions of (s + 3)/(s^2 + 3s + 2).	SOLVED	poles = [-2.0, -1.0], residues = [-1.0, 2.0], count = 2
Find the partial fractions of 1/(s + 1)^2.	INVALID_INPUT	Partial fractions of a rational function (simple poles): repeated poles: this capability lists simple poles only (use th

Convolution integral (f * g)(t) — `convolution_integral`

(fg)(t) = integral from 0 to t of f(u) g(t - u) du, whose transform is F(s) G(s).

$$(f * g)(t) = \int_0^t f(u)\, g(t-u)\, du$$

input	meaning	condition
`f`	function f(t)	
`g`	function g(t)	
`t`	time (>= 0)	

output	meaning
`value`	(fg)(t)

- **Method:** adaptive tanh-sinh quadrature of the convolution integral at 20 digits
- **Assumes:** f, g integrable on [0, t]
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
differential_equations_ai999x.solve('convolution_integral', f='exp(-t)', g='sin(t)', t=2)
# -> value = 0.73039   (SOLVED)
```

```
differential_equations_ai999x.solve('convolution_integral', f='1', g='t', t=2)
# -> value = 2   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the convolution of f(t) = exp(-t) and g(t) = sin(t) at t = 2.	SOLVED	value = 0.73039
Find the convolution of f(t) = exp(-t) and g(t) = sin(t).	NEEDS_INPUT	convolution_integral needs t (time (>= 0)). Example: solve('convolution_integral', f='exp(-t)', g='sin(t)', t=2).

Linear 2x2 system: equilibrium type and stability (trace-determinant plane) — `phase_portrait_2x2`

Classify the origin of x' = A x (node, saddle, spiral, center, star, degenerate) and its stability from the trace and determinant.

$$\lambda^2 - T\lambda + D = 0,\ T = \operatorname{tr} A,\ D = \det A$$

input	meaning	condition
`A`	the 2x2 coefficient matrix of x' = A x	

output	meaning
`type`	phase portrait type
`stability`	stability
`lambda1`	first eigenvalue
`lambda2`	second eigenvalue
`trace`	trace
`det`	determinant

- **Method:** exact trace T, determinant D and $T^2 - 4D$: the trace-determinant plane
- **Assumes:** A real 2x2
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
differential_equations_ai999x.solve('phase_portrait_2x2', A='[[0, 1], [-2, -3]]')
# -> type = stable node, stability = asymptotically stable, lambda1 = -2, lambda2 = -1, trace = -3,
det = 2   (SOLVED)
```

```
differential_equations_ai999x.solve('phase_portrait_2x2', A='[[0, 1], [-1, 0]]')
# -> type = center, stability = stable (not asymptotically), lambda1 = 0 + 1i, lambda2 = 0 - 1i,
trace = 0, det = 1   (SOLVED)
```

```
differential_equations_ai999x.solve('phase_portrait_2x2', A='[[1, 2], [3, 0]]')
# -> type = saddle, stability = unstable, lambda1 = -2, lambda2 = 3, trace = 1, det = -6   (SOLVED)
```

```
differential_equations_ai999x.solve('phase_portrait_2x2', A='[[-1, 0], [0, -1]]')
# -> type = stable star (proper node), stability = asymptotically stable, lambda1 = -1, lambda2 =
-1, trace = -2, det = 1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Classify the equilibrium of the linear system x' = A x with A = [[0, 1], [-2, -3]].	SOLVED	type = stable node, stability = asymptotically stable, lambda1 = -2, lambda2 = -1, trace = -3, det = 2
Phase portrait of x' = A x for A = [[1, -5], [1, 1]].	SOLVED	type = unstable spiral, stability = unstable, lambda1 = 1 + 2.23607i, lambda2 = 1 - 2.23607i, trace = 2, det = 6

question	outcome	result
Phase portrait of x' = A x for A = [[1, 2, 3], [3, 0]].	INVALID_INPUT	A: expected a rectangular matrix (rows of equal length).

Linear system x' = A x: solution at a time — `linear_system_solution`

Solve x' = A x, x(0) = x0 (n up to 6) and give x(t) = e^{At} x0.

$$\mathbf{x}(t) = e^{At}\mathbf{x}_0$$

input	meaning	condition
`A`	n x n coefficient matrix	
`x0`	initial vector x(0)	
`t`	time	

output	meaning
`x`	state x(t)
`x1`	first component of x(t)

- **Method:** matrix exponential at 30 digits (scaling and squaring of the Taylor series)
- **Assumes:** A square, $n \leq 6$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
differential_equations_ai999x.solve('linear_system_solution', A='[[0, 1], [-2, -3]]', x0='1, 0',
t=1)
# -> x = [0.600423599106, -0.46508831587], x1 = 0.600424   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Solve the linear system x' = A x with A = [[0, 1], [-2, -3]], x0 = [1, 0] at t = 1.	SOLVED	x = [0.600423599106, -0.46508831587], x1 = 0.600424
Solve the linear system x' = A x with A = [[0, 1], [-2, -3]], x0 = [1, 0, 2] at t = 1.	INVALID_INPUT	Linear system x' = A x: solution at a time: x0 needs one entry per row of A.

Routh-Hurwitz stability test of a characteristic polynomial — `routh_hurwitz`

How many roots of a polynomial lie in the right half plane (Routh array), hence whether the system is asymptotically stable.

$$\#\{\mathrm{Re}\, s > 0\} = \text{sign changes in the first column}$$

input	meaning	condition
`coeffs`	coefficients, highest power first	

output	meaning
`stable`	all roots in the open left half plane
`rhp_roots`	number of right-half-plane roots
`first_column`	first column of the Routh array

- **Method:** Routh array; sign changes in the first column
- **Assumes:** regular Routh array (no zero in the first column)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
differential_equations_ai999x.solve('routh_hurwitz', coeffs='1, 6, 11, 6')
# -> stable = True, rhp_roots = 0, first_column = [1.0, 6.0, 10.0, 6.0]   (SOLVED)
```

```
differential_equations_ai999x.solve('routh_hurwitz', coeffs='1, 1, 2, 8')
# -> stable = False, rhp_roots = 2, first_column = [1.0, 1.0, -6.0, 8.0]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Apply the Routh-Hurwitz test to coeffs = [1, 6, 11, 6].	SOLVED	stable = True, rhp_roots = 0, first_column = [1.0, 6.0, 10.0, 6.0]
Apply the Routh-Hurwitz test to coeffs = [1, 1, 2, 8].	SOLVED	stable = False, rhp_roots = 2, first_column = [1.0, 1.0, -6.0, 8.0]
Apply the Routh-Hurwitz test to coeffs = [1, 0, 4].	INVALID_INPUT	Routh-Hurwitz stability test of a characteristic polynomial: the Routh array has a zero in its first column (a root on o

Stability of x' = A x (n x n) from the characteristic polynomial — `linear_system_stability`

Whether the origin of x' = A x is asymptotically stable and how many eigenvalues have positive real part, without computing eigenvalues.

$$\det(\lambda I - A) = \lambda^n + c_1\lambda^{n-1} + \cdots + c_n$$

input	meaning	condition
`A`	n x n coefficient matrix $(n \le 8)$	

output	meaning
`stable`	asymptotically stable (all eigenvalues Re < 0)
`unstable_eigenvalues`	number of eigenvalues with $Re > 0$
`char_poly`	characteristic polynomial coefficients

- **Method:** characteristic polynomial by Faddeev-LeVerrier (exact), then the Routh array
- **Assumes:** regular Routh array (no eigenvalue on the imaginary axis)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
differential_equations_ai999x.solve('linear_system_stability', A='[[-1, 2, 0], [0, -3, 1], [1, 0, -2]]')
# -> stable = True, unstable_eigenvalues = 0, char_poly = [1.0, 6.0, 11.0, 4.0]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Is the linear system x' = A x stable for A = [[-1, 2, 0], [0, -3, 1], [1, 0, -2]]?	SOLVED	stable = True, unstable_eigenvalues = 0, char_poly = [1.0, 6.0, 11.0, 4.0]
Is the linear system x' = A x stable for A = [[0, 1], [-1, 0]]?	INVALID_INPUT	Stability of x' = A x (n x n) from the characteristic polynomial: the Routh array has a zero in its first column (a root

Equilibria of y' = f(y) and their stability (phase line) — `autonomous_equilibria`

All equilibrium solutions of an autonomous equation y' = f(y) with polynomial f and their stability from the sign of f.

$$f(y^*) = 0;\ f > 0 \text{ left}, f < 0 \text{ right} \Rightarrow \text{stable}$$

input	meaning	condition
`f`	right-hand side f(y), a polynomial	

output	meaning
`equilibria`	equilibrium values (ascending)
`stability`	stable / unstable / semi-stable for each
`count`	number of equilibria

- **Method:** exact real roots of f; the sign of f between consecutive roots (phase line)
- **Assumes:** f a polynomial in y with rational coefficients, degree <= 8
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
differential_equations_ai999x.solve('autonomous_equilibria', f='y*(1 - y)')
# -> equilibria = [0.0, 1.0], stability = ['unstable', 'stable'], count = 2   (SOLVED)
```

```
differential_equations_ai999x.solve('autonomous_equilibria', f='y^2*(y - 2)')
# -> equilibria = [0.0, 2.0], stability = ['semi-stable', 'unstable'], count = 2   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the equilibria of dy/dt = y*(1 - y) and their stability.	SOLVED	equilibria = [0.0, 1.0], stability = ['unstable', 'stable'], count = 2
Find the equilibria of dy/dt = sin(y) and their stability.	INVALID_INPUT	Equilibria of y' = f(y) and their stability (phase line): f(y) must be a polynomial in y for this capability.

Lotka-Volterra predator-prey: coexistence equilibrium and oscillation period — `lotka_volterra_equilibrium`

For x' = a x - b x y (prey), y' = -c y + d x y (predator): the coexistence equilibrium and the period of small oscillations about it.

$$(x^*, y^*) = \left(\frac{c}{d}, \frac{a}{b}\right),\ T = \frac{2\pi}{\sqrt{ac}}$$

input	meaning	condition
`a`	prey growth rate (> 0)	
`b`	predation rate (> 0)	
`c`	predator death rate (> 0)	
`d`	predator efficiency (> 0)	

output	meaning
`x_eq`	prey at equilibrium c/d
`y_eq`	predators at equilibrium a/b
`omega`	small-oscillation angular frequency sqrt(ac)
`period`	small-oscillation period 2 pi / sqrt(ac)

- **Method:** equilibrium of the nullclines; the Jacobian there has eigenvalues +- i sqrt(ac)
- **Assumes:** a, b, c, $d > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
differential_equations_ai999x.solve('lotka_volterra_equilibrium', a=1, b=0.1, c=1.5, d=0.075)
# -> x_eq = 20, y_eq = 10, omega = 1.22474, period = 5.1302   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Lotka-Volterra predator-prey model with a = 1, b = 0.1, c = 1.5, d = 0.075: find the equilibrium and period.	SOLVED	x_eq = 20, y_eq = 10, omega = 1.22474, period = 5.1302
Lotka-Volterra predator-prey model with a = 1, b = 0.1, c = 1.5: find the equilibrium.	NEEDS_INPUT	lotka_volterra_equilibrium needs d (predator efficiency (> 0)). Example: solve('lotka_volterra_equilibrium', a=1, b=0.1,

Competing species: coexistence equilibrium and its stability — `competing_species_equilibrium`

For x' = x($a_1 - b11\ x - b12$ y), y' = y($a_2 - b21\ x - b22$ y): the coexistence equilibrium and its type (stable coexistence or saddle).

$$J = \begin{pmatrix} -b_{11}x^* & -b_{12}x^* \\ -b_{21}y^* & -b_{22}y^* \end{pmatrix}$$

input	meaning	condition
`a1`	growth rate of x	
`b11`	self-limitation of x	
`b12`	effect of y on x	
`a2`	growth rate of y	
`b21`	effect of x on y	
`b22`	self-limitation of y	

output	meaning
`x_eq`	x at coexistence
`y_eq`	y at coexistence
`type`	equilibrium type
`stability`	stability

- **Method:** solve the linear nullcline equations exactly; classify the Jacobian there by trace and determinant
- **Assumes:** both coexistence values positive; nullclines not parallel
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
differential_equations_ai999x.solve('competing_species_equilibrium', a1=3, b11=1, b12=1, a2=2,
b21=0.5, b22=1)
# -> x_eq = 2, y_eq = 1, type = stable node, stability = asymptotically stable   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Competing species with a1 = 3, b11 = 1, b12 = 1, a2 = 2, b21 = 0.5, b22 = 1: find the coexistence equilibrium.	SOLVED	x_eq = 2, y_eq = 1, type = stable node, stability = asymptotically stable
Competing species with a1 = 1, b11 = 1, b12 = 2, a2 = 3, b21 = 1, b22 = 1: find the coexistence equilibrium.	INVALID_INPUT	Competing species: coexistence equilibrium and its stability: there is no coexistence equilibrium with both populations

Nonlinear 2-D system: type of an equilibrium by linearization — `nonlinear_equilibrium_type`

For x' = f(x, y), y' = g(x, y) and an equilibrium (x0, y0): the Jacobian there, the equilibrium type and stability (Hartman-Grobman).

$$J = \begin{pmatrix} f_x & f_y \\ g_x & g_y \end{pmatrix}_{(x_0, y_0)}$$

input	meaning	condition
`f`	x' = f(x, y)	
`g`	y' = g(x, y)	
`x0`	equilibrium x	

input	meaning	condition
`y0`	equilibrium y	

output	meaning
`type`	type of the linearization
`stability`	stability (inconclusive for a center or a degenerate case)
`jacobian`	Jacobian at the equilibrium
`trace`	trace of the Jacobian
`det`	determinant of the Jacobian

- **Method:** exact symbolic Jacobian at the point; trace-determinant classification
- **Assumes:** (x0, y0) is an equilibrium
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
differential_equations_ai999x.solve('nonlinear_equilibrium_type', f='y', g='-sin(x) - 0.5*y', x0=0, y0=0)
# -> type = stable spiral, stability = asymptotically stable, jacobian = [[0.0, 1.0], [-1.0, -0.5]], trace = -0.5, det = 1   (SOLVED)
```

```
differential_equations_ai999x.solve('nonlinear_equilibrium_type', f='x - x*y', g='x*y - y', x0=1, y0=1)
# -> type = center, stability = linearization inconclusive, jacobian = [[0.0, -1.0], [1.0, 0.0]], trace = 0, det = 1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Classify the equilibrium of the nonlinear system dx/dt = y, dy/dt = -sin(x) - 0.5*y at x0 = 0, y0 = 0.	SOLVED	type = stable spiral, stability = asymptotically stable, jacobian = [[0.0, 1.0], [-1.0, -0.5]], trace = -0.5, det = 1
Classify the equilibrium of the nonlinear system dx/dt = y, dy/dt = -sin(x) - 0.5*y at x0 = 1, y0 = 0.	INVALID_INPUT	Nonlinear 2-D system: type of an equilibrium by linearization: the point is not an equilibrium (f and g do not both vani

Stability function (amplification factor) of a one-step method — `amplification_factor`

For y' = lambda y: the factor R(h lambda) a method multiplies by per step, its size, and whether the step is stable (|R| <= 1).

$$y_{k+1} = R(h\lambda)\, y_k$$

input	meaning	condition
`method`	the one-step method	
`lam`	lambda (real)	
`h`	step size (> 0)	

output	meaning
`R`	amplification factor R(h lambda)
`abs_R`	|R|
`stable`	|R| <= 1
`z`	h lambda

- **Method:** R(z): $1+z$ (Euler), $1+z+z^2/2$ (Heun, midpoint), Taylor to $z^4/24$ (RK4), $1/(1-z)$, $(1+z/2)/(1-z/2)$
- **Assumes:** $h>0$; real lambda
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
differential_equations_ai999x.solve('amplification_factor', method='rk4', lam=-10, h=0.2)
# -> R = 0.333333, abs_R = 0.333333, stable = True, z = -2   (SOLVED)
```

```
differential_equations_ai999x.solve('amplification_factor', method='euler', lam=-10, h=0.25)
# -> R = -1.5, abs_R = 1.5, stable = False, z = -2.5   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the amplification factor of the rk4 method for lam = -10, h = 0.2.	SOLVED	R = 0.333333, abs_R = 0.333333, stable = True, z = -2
Amplification factor of the backward euler method with lam = -50, h = 0.1.	SOLVED	R = 0.166667, abs_R = 0.166667, stable = True, z = -5
Amplification factor of the euler method with lam = -10.	NEEDS_INPUT	amplification_factor needs h (step size (> 0)). Example: solve('amplification_factor', method='rk4', lam=-10, h=0.2).

Largest stable step of an explicit method for y' = lambda y — `explicit_step_limit`

The absolute-stability step limit h_max of Euler, Heun, midpoint or RK4 on a decaying mode y' = lambda y $(\lambda<0)$.

$$|R(h\lambda)|\le 1:\ h\le \frac{x^*}{|\lambda|}\ (x^*=2\ \text{Euler}, 2.785\ \text{RK4})$$

input	meaning	condition
`method`	explicit method	
`lam`	lambda (< 0)	

output	meaning
`h_max`	largest stable step
`z_boundary`	real stability boundary h lambda

- **Method:** bisection for |R(-x)| = 1 on the stability polynomial
- **Assumes:** $\lambda<0$

- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
differential_equations_ai999x.solve('explicit_step_limit', method='euler', lam=-20)
# -> h_max = 0.1, z_boundary = -2   (SOLVED)
```

```
differential_equations_ai999x.solve('explicit_step_limit', method='rk4', lam=-1)
# -> h_max = 2.78529, z_boundary = -2.78529   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Largest stable step of the rk4 method for lam = -20.	SOLVED	h_max = 0.139265, z_boundary = -2.78529
Largest stable step of the euler method for lam = 3.	INVALID_INPUT	Largest stable step of an explicit method for y' = lambda y: needs lambda < 0 (a decaying mode).

CFL condition for the advection equation (upwind scheme) — `cfl_condition`

Courant number C = |c| dt/dx of the explicit upwind scheme for u_t + c $u_x = 0$, its stability $(C \le 1)$ and the largest stable dt.

$$C = \frac{|c|\,\Delta t}{\Delta x} \le 1$$

input	meaning	condition
`c`	advection speed	$c \neq 0$
`dt`	time step (> 0)	
`dx`	grid spacing (> 0)	

output	meaning
`courant`	Courant number |c| dt / dx
`stable`	$C \le 1$
`dt_max`	largest stable time step dx / |c|

- **Method:** Courant-Friedrichs-Lewy condition (von Neumann: $|1 - C(1 - e$^{-i theta})| <= 1)
- **Assumes:** $c \neq 0$; dt, $dx > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
differential_equations_ai999x.solve('cfl_condition', c=2, dt=0.01, dx=0.05)
# -> courant = 0.4, stable = True, dt_max = 0.025   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Check the CFL condition for c = 2, dt = 0.01, dx = 0.05.	SOLVED	courant = 0.4, stable = True, dt_max = 0.025

question	outcome	result
Check the CFL condition for c = 2, dt = 0.01.	NEEDS_INPUT	cfl_condition needs dx (grid spacing (> 0)). Example: solve('cfl_condition', c=2, dt=0.01, dx=0.05).

Stability of the explicit (FTCS) scheme for the heat equation — `explicit_heat_stability`

The mesh ratio $r = \alpha\, dt/dx^2$ of the explicit scheme for $u_t = \alpha$ u_xx, its stability $(r \le 1/2)$ and the largest stable dt.

$$r = \frac{\alpha\,\Delta t}{\Delta x^2} \le \frac{1}{2}$$

input	meaning	condition
`alpha`	diffusivity (> 0)	
`dt`	time step (> 0)	
`dx`	grid spacing (> 0)	

output	meaning
`r`	mesh ratio alpha dt/dx^2
`stable`	$r \le 1/2$
`dt_max`	largest stable time step dx^2/(2 alpha)

- **Method:** von Neumann: $G = 1 - 4$ r sin^2(theta/2), |G| <= 1 iff $r \le 1/2$
- **Assumes:** alpha, dt, $dx > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
differential_equations_ai999x.solve('explicit_heat_stability', alpha=1, dt=0.001, dx=0.05)
# -> r = 0.4, stable = True, dt_max = 0.00125   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Is the explicit FTCS scheme for the heat equation stable with alpha = 1, dt = 0.001, dx = 0.05?	SOLVED	r = 0.4, stable = True, dt_max = 0.00125
Is the explicit FTCS scheme for the heat equation stable with alpha = 1, dt = 0.002, dx = 0.05?	SOLVED	r = 0.8, stable = False, dt_max = 0.00125

Explicit finite-difference (FTCS) steps for the heat equation — `heat_ftcs_steps`

Advance grid values of $u_t = \alpha$ u_xx by n explicit steps u_i <- u_i + r (u_{$i + 1$} - 2 u_i + u_{$i - 1$}), ends held fixed.

$$u_i^{k+1} = u_i^k + r\left(u_{i+1}^k - 2u_i^k + u_{i-1}^k\right)$$

input	meaning	condition
`u`	grid values (first and last are boundary values)	

input	meaning	condition
`r`	mesh ratio alpha dt/dx^2 (> 0)	
`n`	number of time steps	

output	meaning
`u`	grid values after n steps
`stable_ratio`	$r \le 1/2$

- **Method:** forward time, centred space
- **Assumes:** Dirichlet ends (held fixed); $r > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
differential_equations_ai999x.solve('heat_ftcs_steps', u='0, 50, 100, 50, 0', r=0.25, n=1)
# -> u = [0.0, 50.0, 75.0, 50.0, 0.0], stable_ratio = True   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Apply the explicit FTCS heat step to u = [0, 50, 100, 50, 0] with r = 0.25, n = 1.	SOLVED	u = [0.0, 50.0, 75.0, 50.0, 0.0], stable_ratio = True
Apply the explicit FTCS heat step to u = [0, 50] with r = 0.25, n = 1.	INVALID_INPUT	u: needs at least 3 entries, got 2.

Crank-Nicolson steps for the heat equation — `crank_nicolson_steps`

Advance grid values of $u_t = \alpha$ u_xx by n Crank-Nicolson steps (unconditionally stable), ends held fixed.

$$-\tfrac{r}{2}u_{i-1}^{k+1} + (1+r)u_i^{k+1} - \tfrac{r}{2}u_{i+1}^{k+1} = \tfrac{r}{2}u_{i-1}^{k} + (1-r)u_i^{k} + \tfrac{r}{2}u_{i+1}^{k}$$

input	meaning	condition
`u`	grid values (first and last are boundary values)	
`r`	mesh ratio alpha dt/dx^2 (> 0)	
`n`	number of time steps	

output	meaning
`u`	grid values after n steps

- **Method:** average of explicit and implicit steps; each step a tridiagonal solve (Thomas algorithm)
- **Assumes:** Dirichlet ends; $r > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
differential_equations_ai999x.solve('crank_nicolson_steps', u='0, 50, 100, 50, 0', r=1, n=1)
# -> u = [0.0, 35.7142857143, 42.8571428571, 35.7142857143, 0.0]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Apply Crank-Nicolson to u = [0, 50, 100, 50, 0] with r = 1, n = 1.	SOLVED	u = [0.0, 35.7142857143, 42.8571428571, 35.7142857143, 0.0]
Apply Crank-Nicolson to u = [0, 50, 100, 50, 0] with r = 1.	NEEDS_INPUT	crank_nicolson_steps needs n (number of time steps). Example: solve('crank_nicolson_steps', u='0, 50, 100, 50, 0', r=1,

Fourier sine series coefficient (half-range expansion) — `fourier_sine_coefficient`

The coefficient b_n of the sine series of f(x) on [0, L], used for the heat and wave equations with fixed ends.

$$b_n = \frac{2}{L}\int_0^L f(x)\sin\frac{n\pi x}{L}\,dx$$

input	meaning	condition
`f`	initial profile f(x), a polynomial	
`L`	length of the interval (> 0)	
`n`	index $n \geq 1$	

output	meaning
`b_n`	sine coefficient b_n

- **Method:** exact integration by parts of x^k sin(n pi x / L)
- **Assumes:** f a polynomial; $L > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
differential_equations_ai999x.solve('fourier_sine_coefficient', f='x*(1 - x)', L=1, n=1)
# -> b_n = 0.258012   (SOLVED)
```

```
differential_equations_ai999x.solve('fourier_sine_coefficient', f='1', L=1, n=2)
# -> b_n = 0   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the Fourier sine coefficient of f(x) = x*(1 - x) with L = 1, n = 1.	SOLVED	b_n = 0.258012
Find the Fourier sine coefficient of f(x) = sin(x) with L = 1, n = 1.	INVALID_INPUT	Fourier sine series coefficient (half-range expansion): this capability needs f(x) to be a polynomial in x.

Heat equation on a rod with ends at zero: Fourier series value — `heat_equation_series`

u(x, t) for $u_t = \alpha$ u_xx, u(0, t) = u(L, t) = 0, u(x, 0) = f(x) (a polynomial), summed over the first N series terms.

$$u(x,t) = \sum_{n=1}^{N} b_n e^{-\alpha(n\pi/L)^2 t}\sin\frac{n\pi x}{L}$$

input	meaning	condition
`f`	initial profile f(x), a polynomial	
`L`	rod length (> 0)	
`alpha`	diffusivity (> 0)	
`N`	number of series terms (1 to 500)	
`x`	position $(0 \le x \le L)$	
`t`	time (>= 0)	

output	meaning
`u`	temperature u(x, t) from N terms
`b_1`	first sine coefficient

- **Method:** separation of variables: u = sum b_n e^{-alpha (n pi/L)^2 t} sin(n pi x/L)
- **Assumes:** f a polynomial; Dirichlet ends at 0; series truncated after N terms (stated)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
differential_equations_ai999x.solve('heat_equation_series', f='x*(1 - x)', L=1, alpha=1, N=10,
x=0.5, t=0.1)
# -> u = 0.0961619, b_1 = 0.258012   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Heat equation series with f(x) = x*(1 - x), L = 1, alpha = 1, N = 10, x = 0.5, t = 0.1.	SOLVED	u = 0.0961619, b_1 = 0.258012
Heat equation series with f(x) = x*(1 - x), L = 1, alpha = 1, N = 10, x = 2, t = 0.1.	INVALID_INPUT	Heat equation on a rod with ends at zero: Fourier series value: needs L > 0, alpha > 0, t >= 0, 0 <= x <= L and 1 <= N <

Vibrating string with fixed ends: Fourier series value — `wave_equation_string_series`

u(x, t) for $u_t t = c^2$ u_xx, u(0, t) = u(L, t) = 0, u(x, 0) = f(x), u_t(x, 0) = g(x) (polynomials), from N series terms.

$$u = \sum_{n=1}^{N} \left[b_n \cos \frac{n\pi ct}{L} + \frac{B_n L}{n\pi c} \sin \frac{n\pi ct}{L} \right] \sin \frac{n\pi x}{L}$$

input	meaning	condition
`f`	initial profile f(x), a polynomial	
`L`	string length (> 0)	
`c`	wave speed (> 0)	
`N`	number of series terms	
`x`	position $(0 \le x \le L)$	
`t`	time	
`g`	initial velocity g(x), a polynomial (default 0)	

output	meaning
`u`	displacement u(x, t) from N terms

- **Method:** u = sum [b_n cos(n pi c t/L) + B_n L/(n pi c) sin(n pi c t/L)] sin(n pi x/L)
- **Assumes:** polynomial f, g; fixed ends; N terms (stated)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
differential_equations_ai999x.solve('wave_equation_string_series', f='x*(1 - x)', L=1, c=2, N=20,
x=0.25, t=0.3)
# -> u = -0.0499929   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Vibrating string with f(x) = x*(1 - x), L = 1, c = 2, N = 20, x = 0.25, t = 0.3.	SOLVED	u = -0.0499929
Vibrating string with f(x) = x*(1 - x), L = 1, c = 2, N = 20, x = 0.25.	NEEDS_INPUT	wave_equation_string_series needs t (time). Example: solve('wave_equation_string_series', f='x*(1 - x)', L=1, c=2, N=20,

Wave equation on the whole line: d'Alembert's formula — `dalembert_solution`

u(x, t) for $u_tt = c^2$ u_xx on the real line with u(x, 0) = f(x), u_t(x, 0) = g(x).

$$u(x,t) = \frac{f(x-ct) + f(x+ct)}{2} + \frac{1}{2c}\int_{x-ct}^{x+ct} g(s)\,ds$$

input	meaning	condition
`f`	initial displacement f(x)	
`g`	initial velocity g(x)	
`c`	wave speed (> 0)	
`x`	position	
`t`	time	

output	meaning
`u`	displacement u(x, t)

- **Method:** u = [f(x - ct) + f(x + ct)]/2 + (1/2c) * integral of g over [x - ct, x + ct] (tanh-sinh quadrature)
- **Assumes:** $c > 0$; g integrable
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
differential_equations_ai999x.solve('dalembert_solution', f='sin(x)', g='0', c=2, x=1, t=0.5)
# -> u = 0.454649   (SOLVED)
```

```
differential_equations_ai999x.solve('dalembert_solution', f='0', g='1', c=2, x=1, t=0.5)
# -> u = 0.5   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Use d'Alembert's formula with f(x) = sin(x), g(x) = 0, c = 2, x = 1, t = 0.5.	SOLVED	u = 0.454649
Use d'Alembert's formula with f(x) = sin(x), g(x) = 0, c = -2, x = 1, t = 0.5.	INVALID_INPUT	Wave equation on the whole line: d'Alembert's formula: the wave speed c must be positive.

Laplace's equation on a rectangle: series value — `laplace_equation_rectangle`

Steady state $u_x x + u_y y = 0$ on [0, a] x [0, b] with $u = f$(x) on the top edge $y = b$ and 0 on the other three edges, from N terms.

$$u(x,y) = \sum_{n=1}^{N} b_n \frac{\sinh(n\pi y/a)}{\sinh(n\pi b/a)} \sin\frac{n\pi x}{a}$$

input	meaning	condition
`f`	top-edge values f(x), a polynomial	
`a`	width (> 0)	
`b`	height (> 0)	
`N`	number of series terms	
`x`	x in [0, a]	
`y`	y in [0, b]	

output	meaning
`u`	u(x, y) from N terms

- **Method:** u = sum b_n sinh(n pi y/a)/sinh(n pi b/a) sin(n pi x/a)
- **Assumes:** polynomial f; zero on three edges; N terms (stated)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
differential_equations_ai999x.solve('laplace_equation_rectangle', f='100', a=1, b=1, N=50, x=0.5,
y=0.5)
# -> u = 25   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Solve Laplace's equation on a rectangle with top edge f(x) = 100, a = 1, b = 1, N = 50, x = 0.5, y = 0.5.	SOLVED	u = 25
Solve Laplace's equation on a rectangle with top edge f(x) = 100, a = 1, b = 1, N = 50, x = 0.5, y = 2.	INVALID_INPUT	Laplace's equation on a rectangle: series value: needs a, b > 0, (x, y) in the rectangle and 1 <= N <= 500.

Eigenvalues of y'' + lambda y = 0 with Dirichlet/Neumann ends — `sturm_liouville_eigenvalues`

The n-th eigenvalue and eigenfunction of y'' + lambda $y = 0$ on [0, L] with Dirichlet $(y = 0)$ or Neumann (y' = 0) conditions at each end.

$$\lambda_n = \left(\frac{n\pi}{L}\right)^2 \text{(DD, NN)}, \quad \left(\frac{(n-\frac{1}{2})\pi}{L}\right)^2 \text{(mixed)}$$

input	meaning	condition
`bc`	boundary conditions at $x = 0$ and $x = L$	
`L`	interval length (> 0)	
`n`	index $n \geq 1$ (Neumann-Neumann also has lambda_0 = 0)	

output	meaning
`lambda_n`	n-th eigenvalue
`sqrt_lambda`	sqrt(lambda_n)
`eigenfunction`	eigenfunction

- **Method:** sin/cos solutions; the boundary conditions quantise sqrt(lambda)
- **Assumes:** $L > 0$; $n \geq 1$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
differential_equations_ai999x.solve('sturm_liouville_eigenvalues', bc='dirichlet-dirichlet', L=1,
n=2)
# -> lambda_n = 39.4784, sqrt_lambda = 6.28319, eigenfunction = sin(6.28318530718 x)   (SOLVED)
```

```
differential_equations_ai999x.solve('sturm_liouville_eigenvalues', bc='dirichlet-neumann', L=1,
n=1)
# -> lambda_n = 2.4674, sqrt_lambda = 1.5708, eigenfunction = sin(1.57079632679 x)   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the eigenvalue of y'' + lambda y = 0 with dirichlet-dirichlet conditions, L = 1, n = 2.	SOLVED	lambda_n = 39.4784, sqrt_lambda = 6.28319, eigenfunction = sin(6.28318530718 x)
Find the eigenvalue of y'' + lambda y = 0 with dirichlet-dirichlet conditions, L = -1, n = 2.	INVALID_INPUT	Eigenvalues of y'' + lambda y = 0 with Dirichlet/Neumann ends: needs L > 0 and n >= 1.

Eigenvalues with a Robin (convective) end condition — `robin_eigenvalues`

The n-th eigenvalue of y'' + lambda $y = 0$, y(0) = 0, y'(L) + h y(L) = 0 (heat loss at one end): root of tan(mu L) = -mu/h.

$$\tan(\mu L) = -\frac{\mu}{h}, \quad \lambda = \mu^2$$

input	meaning	condition
`L`	interval length (> 0)	
`h`	Robin coefficient (> 0)	
`n`	index $n \geq 1$	

output	meaning
`lambda_n`	n-th eigenvalue mu_n^2
`sqrt_lambda`	mu_n

- **Method:** bisection for mu cos(mu L) + h sin(mu L) = 0 in $((n - 1/2)$ pi/L, n pi/L)
- **Assumes:** $L > 0$; $h > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
differential_equations_ai999x.solve('robin_eigenvalues', L=1, h=1, n=1)
# -> lambda_n = 4.11586, sqrt_lambda = 2.02876   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the Robin eigenvalue with L = 1, h = 1, n = 1.	SOLVED	lambda_n = 4.11586, sqrt_lambda = 2.02876
Find the Robin eigenvalue with L = 1, h = -1, n = 1.	INVALID_INPUT	Eigenvalues with a Robin (convective) end condition: needs L > 0, h > 0 and n >= 1.

Logistic map: fixed points and their stability — `logistic_map_fixed_points`

Fixed points of x -> r x $(1 - x)$ in [0, 1], their multipliers f'(x*) and stability.

$$x^* = 1 - \frac{1}{r}, \quad f'(x^*) = 2 - r$$

input	meaning	condition
`r`	parameter $(0 < r \leq 4)$	

output	meaning
`fixed_points`	fixed points in [0, 1]
`multipliers`	f'(x*) at each
`stability`	stable / unstable / neutral
`count`	number of fixed points

- **Method:** x* = 0 and $1 - 1/r$; multipliers r and $2 - r$
- **Assumes:** $0 < r \leq 4$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
differential_equations_ai999x.solve('logistic_map_fixed_points', r=2.5)
# -> fixed_points = [0.0, 0.6], multipliers = [2.5, -0.5], stability = ['unstable', 'stable'],
count = 2   (SOLVED)
```

```
differential_equations_ai999x.solve('logistic_map_fixed_points', r=3.2)
# -> fixed_points = [0.0, 0.6875], multipliers = [3.2, -1.2], stability = ['unstable', 'unstable'],
count = 2   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the fixed points of the logistic map with r = 2.5.	SOLVED	fixed_points = [0.0, 0.6], multipliers = [2.5, -0.5], stability = ['unstable', 'stable'], count = 2
Find the fixed points of the logistic map with r = 5.	INVALID_INPUT	Logistic map: fixed points and their stability: the logistic map x -> r x (1 - x) needs 0 < r <= 4.

Logistic map: period-2 cycle and its stability — `logistic_map_two_cycle`

The 2-cycle {p, q} of x -> r x $(1 - x)$ $(3 < r \leq 4)$, its multiplier f'(p) f'(q) and stability (stable for $3 < r < 1 + sqrt$ 6).

$$p, q = \frac{r + 1 \mp \sqrt{(r-3)(r+1)}}{2r}, \quad f'(p)f'(q) = 4 + 2r - r^2$$

input	meaning	condition
`r`	parameter $(3 < r \leq 4)$	

output	meaning
`p`	lower cycle point
`q`	upper cycle point
`multiplier`	f'(p) f'$(q) = 4 + 2r - r^2$
`stability`	stability

- **Method:** roots of f(f(x)) = x other than the fixed points
- **Assumes:** $3 < r \leq 4$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
differential_equations_ai999x.solve('logistic_map_two_cycle', r=3.2)
# -> p = 0.513045, q = 0.799455, multiplier = 0.16, stability = stable   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the period-2 cycle of the logistic map with r = 3.2.	SOLVED	p = 0.513045, q = 0.799455, multiplier = 0.16, stability = stable
Find the period-2 cycle of the logistic map with r = 2.5.	INVALID_INPUT	Logistic map: period-2 cycle and its stability: a 2-cycle exists for 3 < r <= 4.

Lyapunov exponent of the logistic map (periodic regime) — `logistic_map_lyapunov`

The Lyapunov exponent of x -> r x $(1 - x)$: the average of ln|f'(x_k)| over n iterations after the orbit settles (fixed point or 2-cycle).

$$\lambda = \frac{1}{n} \sum_{k=1}^{n} \ln \left| r(1 - 2x_k) \right|$$

input	meaning	condition
`r`	parameter $(0 < r \leq 4)$	
`x0`	initial value in (0, 1)	
`n`	iterations averaged (even)	

output	meaning
`lyapunov`	Lyapunov exponent (negative: stable periodic orbit)
`attractor`	attractor reached

- **Method:** iterate until settled, then average ln|r (1 - 2 x_k)| over n iterations
- **Assumes:** the orbit settles on a fixed point or 2-cycle (|x_{$k + 2$} - x_k| < 1e-14); n even
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
differential_equations_ai999x.solve('logistic_map_lyapunov', r=2.5, x0=0.3, n=1000)
# -> lyapunov = -0.693147, attractor = fixed point   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the Lyapunov exponent of the logistic map with r = 2.5, x0 = 0.3, n = 1000.	SOLVED	lyapunov = -0.693147, attractor = fixed point
Find the Lyapunov exponent of the logistic map with r = 3.9, x0 = 0.3, n = 1000.	INVALID_INPUT	Lyapunov exponent of the logistic map (periodic regime): the orbit does not settle on a fixed point or a 2-cycle (chaoti

Lorenz system: equilibria and stability of the origin — `lorenz_equilibria`

Equilibria of x' = sigma(y - x), y' = x(rho - z) - y, z' = xy - beta z: the origin and C+- = (+-sqrt(beta(rho-1)), same, rho - 1).

$$C_{\pm} = \left(\pm\sqrt{\beta(\rho - 1)}, \pm\sqrt{\beta(\rho - 1)}, \rho - 1 \right)$$

input	meaning	condition
`sigma`	Prandtl number (> 0)	
`rho`	Rayleigh parameter (> 0)	
`beta`	geometric factor (> 0)	

output	meaning
`count`	number of equilibria
`x_c`	x (and y) of C+
`z_c`	z of C+- $(\rho - 1)$
`origin_stability`	stability of the origin

- **Method:** nullclines; the origin's Jacobian has eigenvalue signs set by $\rho - 1$
- **Assumes:** sigma, rho, $\beta > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
differential_equations_ai999x.solve('lorenz_equilibria', sigma=10, rho=28, beta=2.6666666666666665)
# -> origin_stability = unstable, count = 3, x_c = 8.48528, z_c = 27   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the equilibria of the Lorenz system with sigma = 10, rho = 28, beta = 8/3.	SOLVED	origin_stability = unstable, count = 3, x_c = 8.48528, z_c = 27
Find the equilibria of the Lorenz system with sigma = 10, rho = 28.	NEEDS_INPUT	lorenz_equilibria needs beta (geometric factor (> 0)). Example: solve('lorenz_equilibria', sigma=10, rho=28, beta=2.6666

Lorenz system: Hopf threshold rho_H (loss of stability of C+-) — `lorenz_hopf_threshold`

The Rayleigh parameter at which the equilibria C+- of the Lorenz system lose stability (subcritical Hopf; chaos follows).

$$\rho_H = \frac{\sigma(\sigma + \beta + 3)}{\sigma - \beta - 1}$$

input	meaning	condition
`sigma`	Prandtl number (> beta + 1)	
`beta`	geometric factor (> 0)	

output	meaning
`rho_H`	Hopf threshold

- **Method:** Routh-Hurwitz on the characteristic cubic at C+-
- **Assumes:** $\sigma > \beta + 1$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
differential_equations_ai999x.solve('lorenz_hopf_threshold', sigma=10, beta=2.6666666666666665)
# -> rho_H = 24.7368   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the Hopf threshold of the Lorenz system with sigma = 10, beta = 8/3.	SOLVED	rho_H = 24.7368
Find the Hopf threshold of the Lorenz system with sigma = 2, beta = 8/3.	INVALID_INPUT	Lorenz system: Hopf threshold rho_H (loss of stability of C+-): needs sigma > beta + 1 (otherwise C+- never lose stabili

Brusselator: equilibrium, its type, and the Hopf value b = 1 + a^2 — `brusselator_hopf`

For x' = $a - (b+1)\,x + x^2$ y, y' = b $x - x^2$ y: the equilibrium (a, b/a), its type and the critical b where a limit cycle is born.

$$b_c = 1 + a^2$$

input	meaning	condition
`a`	parameter a (> 0)	
`b`	parameter b (> 0)	

output	meaning
`x_eq`	x at equilibrium
`y_eq`	y at equilibrium
`b_critical`	Hopf bifurcation value $1 + a^2$
`type`	equilibrium type
`stability`	stability

- **Method:** Jacobian [[$b - 1$, a^2], [-b, $-a^2$]]: trace $b - 1 - a^2$ changes sign at the Hopf point
- **Assumes:** a, $b > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
differential_equations_ai999x.solve('brusselator_hopf', a=1, b=3)
# -> x_eq = 1, y_eq = 3, b_critical = 2, type = unstable spiral, stability = unstable   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Analyse the Brusselator with a = 1, b = 3.	SOLVED	x_eq = 1, y_eq = 3, b_critical = 2, type = unstable spiral, stability = unstable
Analyse the Brusselator with a = 1, b = -3.	INVALID_INPUT	Brusselator: equilibrium, its type, and the Hopf value b = 1 + a^2: needs a > 0 and b > 0.

SIR epidemic: R0, peak infections and final size — `sir_epidemic`

For S' = -beta S I, I' = beta S I - gamma I (fractions of the population): R0, the peak infected fraction and the final susceptible fraction.

$$I_{\max} = S_0 + I_0 - \frac{1 + \ln(R_0 S_0)}{R_0}, \quad \ln\frac{S_\infty}{S_0} = R_0(S_\infty - S_0 - I_0)$$

input	meaning	condition
`beta`	transmission rate (> 0)	
`gamma`	recovery rate (> 0)	
`S0`	initial susceptible fraction	
`I0`	initial infected fraction	

output	meaning
`R0`	basic reproduction number beta/gamma
`I_max`	peak infected fraction
`S_inf`	final susceptible fraction
`attack_rate`	fraction infected during the epidemic S0 - S_inf

- **Method:** first integrals: I + S - ln(S)/R0 is conserved; peak at $S = 1/R_0$; final size from ln(S/S0) = R0 $(S - S_0 - I_0)$
- **Assumes:** S0, $I_0 > 0$; $S_0 + I_0 \leq 1$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
differential_equations_ai999x.solve('sir_epidemic', beta=0.3, gamma=0.1, S0=0.99, I0=0.01)
# -> R0 = 3, I_max = 0.303813, S_inf = 0.0587974, attack_rate = 0.931203   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
SIR epidemic model with beta = 0.3, gamma = 0.1, S0 = 0.99, I0 = 0.01: find the peak and final size.	SOLVED	R0 = 3, I_max = 0.303813, S_inf = 0.0587974, attack_rate = 0.931203
SIR epidemic model with beta = 0.3, gamma = 0.1, S0 = 0.99: find the peak.	NEEDS_INPUT	sir_epidemic needs I0 (initial infected fraction). Example: solve('sir_epidemic', beta=0.3, gamma=0.1, S0=0.99, I0=0.01)